

第 13 讲 VLA 前沿：跑得更快、记得更久、用上全身

目录

1 实时推理：让大模型跟上控制频率	2
1.1 为什么“实时”是 VLA 落地的第一个卡点	2
1.2 三条路线：算法 / 系统 / 学习 + 投机	3
1.3 RTC：把“等推理”变成“边算边接”	4
1.4 FASTER：不只不卡，还要反应快	7
1.5 Realtime-VLA V1：把模型本身榨到 30 FPS	9
1.6 Realtime-VLA V2：学会何时快、何时慢	12
1.7 Realtime-VLA FLASH：用投机推理少算几次	14
1.8 Legato：把续接从推理时的补丁变成训练时学到的属性	16
1.9 本节小结	19
2 长时记忆：让 VLA 记得住、想得到	20
2.1 单帧 VLA 的马尔可夫假设为什么不够	20
2.2 一条主线：从隐式上下文到记忆 + 想象	20
2.3 Long-Context DP：先问“能不能就把历史喂进去”	21
2.4 HAMLET：给现成 VLA 外挂一个轻记忆	22
2.5 记忆要“找得回、用得上”：Chameleon 的三条标尺	23
2.6 MemoryVLA：把记忆做成显式记忆库	24
2.7 Dual-Memory VLA：记忆还能用来提速和稳住生成	27
2.8 从“记住过去”到“预判未来”：MemoryVLA++	27
2.9 BagelVLA：另一条向前看的路线	28
2.10 Long-VLA：把长程能力直接收口	29
2.11 易混点：“记忆”和“想象”各指好几样东西	30
2.12 记忆的代价，以及“收益是在哪量出来的”	30
2.13 本节小结	31
3 语言直驱全身：让一句话落到整个身体	32
3.1 人形全身难在哪	32
3.2 LangWBC：先证明语言能直驱全身	32

3.3 LeVERB：高层看图、低层稳执行	36
3.4 WholeBodyVLA：移动服务操作	40
3.5 Humanoid-VLA：数据规模化	43
3.6 本节小结	45
4 全身底座与基础模型：让身体先会走稳	46
4.1 HOVER：立底座	46
4.2 SONIC：做大底座	49
4.3 GR00T N1：强大脑，但没有腿	52
4.4 GR00T N1.5 / N1.6：用解耦全身控制把两条线合龙	55
4.5 本节小结	57
5.1 三条前沿的共同方向	58
5.2 读一篇 VLA 前沿论文，先问四句	58
5.3 下一讲：换范式进入强化学习	59
参考文献	59

第 12 讲我们把一个 VLA 微调到了自己的机器人上（那一讲 2.3 节把 SmolVLA 全量微调到 SO-101 仿真），让它在桌面任务里能跑、能用。可“能用”和“能落地”之间还隔着三道坎：它**太慢**，跟不上控制频率；它**记不住**，长程任务做着做着就乱；它**只有一只手**，没法用整个人形身体去干活。

这三道坎各自都已经长出一条活跃的研究线：实时推理让 VLA 跑得更快，长时记忆让它记得更久，人形全身 VLA 让它用上整个身体。第 1、2 节分别对着前两道坎；第三道坎要分两节，因为“用上整个身体”这件事从一开始就是两端在往中间修——第 3 节看**大脑那一端**怎么把一句话落到全身动作，第 4 节看**身体这一端**怎么先会走稳、再被做成通用底座。每节先摆出这条线的地图，再挑几篇代表作看清它们各自解决了什么、又付出了什么代价。这一讲没有配套实验，读完不会多一个能跑的模型，但会多一把判断“下一个 VLA 值不值得跟”的尺子。

1 实时推理：让大模型跟上控制频率

1.1 为什么“实时”是 VLA 落地的第一个卡点

当前主流大 VLA 推理一次——图像编码 + 语言模型 prefill + 动作去噪——完整跑一轮往往要 58–106 ms，还不含网络通信。这个区间的两端后面都会给出处：58 ms 是 1.6 节 FLASH 那篇在 LIBERO 上测到的一轮完整推理，106 ms 是 1.5 节那篇用朴素 PyTorch 实现、在单张消费级显卡上双视角跑出来的数。这个数字本身不算“慢”，要判断它够不够快，得放到机器人自己的时间预算里去比：

指标	含义
首动作延迟	从拿到新观测，到第一个可执行动作发出去，隔了多久
感知到动作端到端延迟	传感器采样 → 预处理 → 推理 → 后处理 → 下发的全链路耗时
闭环重规划周期	多久重新看一眼环境、重新规划一次
动作块边界停顿	一个块执行完、下一个块还没算好时，机器人空等多久
任务允许时限	这个任务留给你的机会窗口有多长（抓传送带上的物体 vs 慢速摆放，差好几个数量级）

拿这套口径去量，问题就具体了：

- **停顿**：推理还没算完，机器人只能等，在动作块边界出现可见停顿，动态任务直接失败。
- **抖动**：相邻两次规划可能选不同路径，切换点剧烈震动，触发保护停机。
- **反应盲区**：就算消除了停顿，感知到新信息后仍要等整个推理流程跑完才能给出第一个动作——几十毫秒的首动作延迟，在乒乓球、传送带分拣这类任务里足以让机会窗口关闭。

一句话：**大 VLA 单次推理几十到上百毫秒，而真实控制要高频、低延迟、快反应**。这不是调参能抹平的矛盾，需要从算法、系统、学习三个层面协同突破。

备注：常见的一种类比是“人的反应时间约 150–200 ms，所以模型 58–106 ms 已经够快（或还不够快）”。这个对比其实不成立：人的反应时间包含感知、决策与肌肉启动的完整链路，而 VLA 那个数字只是推理这一段，两者不是同一个指标，不能直接比大小。要谈够不够快，就用上表这些机器人侧的量。

1.2 三条路线：算法 / 系统 / 学习 + 投机

同一个“实时”目标，有三种正交的解法。先把地图立起来：

路线	关键问题	代表工作	核心手段
算法层	推理没算完怎么执行不停顿、不抖动？感知到变化怎么快速反应？续接能不能学进策略？	RTC、FASTER、Legato	改动作块怎么生成与衔接；不碰模型骨架
系统层	模型本身太慢，怎么进 30 ms 以内？	Realtime-VLA V1	kernel 工程 + 并发流水线，榨干硬件
学习 + 投机层	跑快了不稳不准怎么办？重复规划能否少推理几次？	Realtime-VLA V2、FLASH	学习速度自适应；投机推理跳过昂贵去噪

三条路线互补而非竞争。算法层大多免训练、即插即用（Legato 要重训一次，FASTER 要一次轻量微调），代价是模型本身该多慢还是多慢；系统层是底层使能，没有它把单次推理压下去，上面两层都无

从谈起，代价是要为具体硬件手写 kernel；学习 + 投机层在“已经快”的基础上把平均延迟压到极限，代价是多训一个模型、多一条回退路径。

上表说清了“谁是谁、用什么手段”，但真正要紧的是**它们动的不是同一段**——把三条路线放回“观测进来 → VLM prefill → N 步去噪 → 动作块出队 → 控制器逐个消费”这条链上，各自的落点是：

- **系统层动的是 prefill 与去噪这两段本身。**1.5 节的 Full Streaming Inference 让大模型慢回路几十毫秒看一次场景、Action Expert 快回路约 2 ms 出一次动作，两条并发跑，单次推理从 106 ms 压到 27 ms。这两段被压短了，链路的其余部分一个字没改。
- **算法层动的是段与段之间的调度。**FASTER 把去噪步数沿时间视界分配，近端一步、远端多步（1.4 节）；RTC 让新算出来的一块与上一块重叠，重叠区切成冻结 / 软过渡 / 自由重画三段（1.3 节）。两者都不改模型权重，只改推理期什么时候算什么。
- **学习 + 投机层根本不在主干上。**FLASH 走旁路：小模型 7.8 ms 猜一整块，主模型并行验证，猜中就绕开去噪、猜错才回主干走完整的 58 ms（1.7 节）。同一层里的 V2 更靠外，它不绕路，学的是“何时快、何时慢”，调的是整条链的节奏。

正因为三者动的不是同一段，把它们叠起来不会互相抵消。1.3 到 1.8 各取其中一条，看它到底在跟什么较劲；其中 1.8 的 Legato 是算法层里唯一一篇把续接**学进训练**的，读完 1.3 再看它对照最清楚。

1.3 RTC：把“等推理”变成“边算边接”

解决什么问题？第一道坎是停顿与抖动。以 $\pi_{0.5}$ 为例：RTC 论文 [1] 在 50 Hz 控制、5 步去噪的设定下量到，一次**模型推理**要 76 ms，而底层控制器每 20 ms 就要消费一个动作（即**动作执行周期** 20 ms，不是每 20 ms 重新推理一次）。一次推理占掉将近四个控制步。动作队列能按 20 ms 稳定出队，靠的是一次推理生成了一整块动作；可一旦这一块执行到底、下一块还没算出来，“算完再动”就意味着每个动作块边界都卡顿。

放进一个具体任务里更清楚。论文用的是“划火柴点蜡烛”：抓起火柴和火柴盒、划燃火柴、点燃蜡烛。策略每次规划 50 步（1 秒）的动作块，执行到一半（25 步、0.5 秒）时开始推理下一块。这次推理要是没能在剩下半块执行完之前算完，机器人就得在块末停下等。停顿的代价不只是慢：动态环境里的状态在这段时间已经变了（火柴的惯性、物体的滑落趋势），恢复执行时策略面对的观测已经偏离训练分布。

抖动的成因更隐蔽。flow / diffusion 策略天生是多模态的，同一个观测下它可以给出好几条都对的路径。于是前一块规划“从障碍物上方绕过去”、后一块在几乎相同的状态下规划“从下方绕”，这叫**分叉 (bifurcation)**。

图里黑色轨迹是当前块、红色是新块，新块要等推理延迟 $d = 7$ 步之后才可用。朴素异步执行在 a_{10} 切到 a'_{11} 处产生一个巨大加速度，足以触发保护停机；Temporal Ensembling 把两条路径取平均，峰值确实降下来了，可平均出来的那条从两条路径中间穿过、正好撞上障碍物，同样落在分布外。**直接拼接和简单平均都不行**，这是 RTC 要绕开的坑。

怎么做的？RTC 的核心洞察可以类比为**图像修复 (inpainting)**：一张照片左半部分已经冲印出来、改不了，右半部分还在底片上、可以重新曝光，目标是让右半和左半天衣无缝地接上。动作序列上的对应

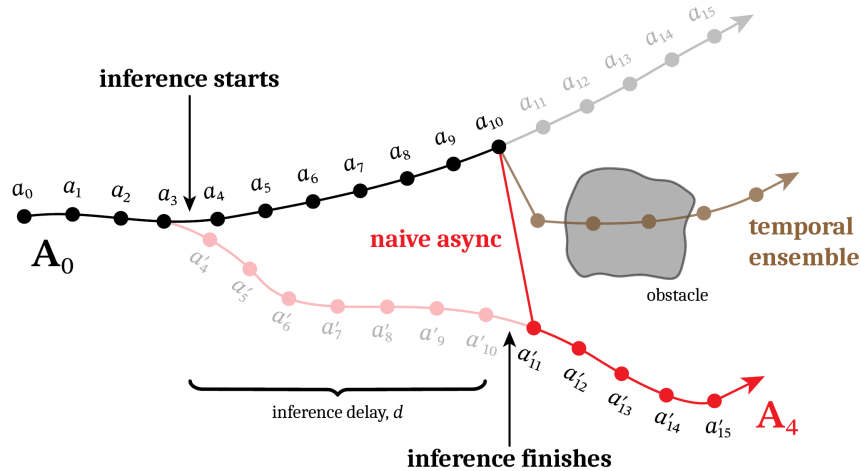


图 1: 朴素异步在切换点猛地一跳，取平均又会穿过障碍物（图片出自参考文献 1）

关系是：已经在执行、来不及改的动作对应已冲印那半，推理延迟内将要执行的动作对应马上要冲印的部分，新块的未来动作对应还在底片上的部分。

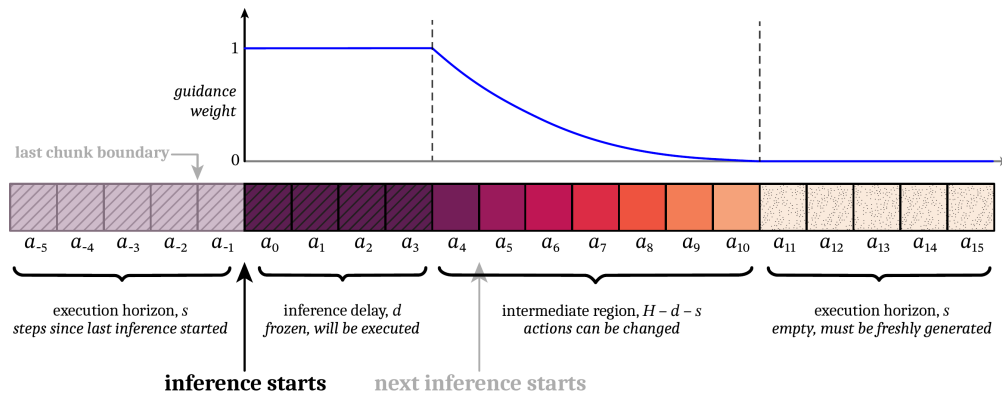


图 2: 把一个动作块切成“冻结 / 软过渡 / 自由重画”三段，只重画未来段，接缝平滑（图片出自参考文献 1）

三个区域的划分要用到三个量：预测视界 H （一块有多长， $\pi_{0.5}$ 取 50 步）、执行视界 s （每次实际执行多少步，约 $H/2$ ）、推理延迟 d （推理占掉几个控制步，LAN 下 76 ms 除以 20 ms 约 6 步）。新块的 H 步动作按这三个量切开：

区域	范围	怎么处理	为什么
冻结区	第 0 到 $d - 1$ 步	锁成上一块的对应值，不参与去噪	这 d 步在推理期间必须执行，已成事实
软掩码区	第 d 到 $H - s - 1$ 步	按指数衰减的权重引导去噪，越靠近冻结区权重越大	有重叠信息可参考，但越往后不确定性越大
自由区	第 $H - s$ 到 $H - 1$ 步	完全自由去噪，权重为 0	超出上一块末尾，没有参考，让模型自己发挥

软掩码区的权重按指数衰减：

$$\mathbf{W}_i = \begin{cases} 1 & i < d \\ c_i \frac{e^{c_i} - 1}{e - 1} & d \leq i < H - s \\ 0 & i \geq H - s \end{cases} \quad c_i = \frac{H - s - i}{H - s - d + 1}$$

这个式子不必记，要记的是它的形状：越靠“已执行”那端，对上一块的引导越强、权重趋向 1；越靠未来那端引导越弱、权重趋向 0，给模型留出适应新观测的余地。

备注：论文把 $\mathbf{W}$ 设成实数向量（软掩码），朴素 inpainting 用的是 0/1 二值掩码（硬掩码）。实验里软掩码在延迟较小时明显更好，因为硬掩码给的约束太弱，新块照样会跳到另一个模式上去。

有了权重和上一块的参考值 $\mathbf{A}_{\text{prev}}$ ，引导去噪的每一步在标准 flow matching 速度场上加一个“拉力”：

$$\mathbf{v}_{\text{RTC}} = \mathbf{v}_{\pi} + \min\left(\beta, \frac{1 - \tau}{\tau \cdot r_{\tau}^2}\right) \cdot \mathbf{g}$$

$\mathbf{g}$ 是加权误差对当前去噪状态的梯度，靠反向传播（VJP，向量-Jacobian 乘积）算出来。这个拉力把新块的预测往 $\mathbf{A}_{\text{prev}}$ 的方向拽，越靠近冻结区拽得越紧，于是新块在冻结区边界处平滑接上。

备注：这套引导机制来自 IIGDM (Pseudoinverse Guidance for Diffusion Models)，原本是给图像 inpainting 用的。RTC 把它移到动作序列上，并加了权重裁剪 β ，防止少步去噪时数值发散。

执行侧，RTC 把推理和执行拆成两个线程。控制线程每 20 ms 取一步动作执行，永远有动作可取；推理线程在后台连续循环，每算完一块就替换掉共享的“当前块”，两者靠一个互斥锁通信。这里有一个容易漏掉的设计：推理线程会**自适应估计下一次的推理延迟 d** ，取最近几次延迟的最大值，故意估保守。估小了就会出现“新块还没算好、冻结区却已经过期”，前面那套三区域划分随之失效。有了这个保守估计，就算推理突然变慢（网络抖动、GPU 被别的进程占了），控制线程也一直有动作输出。

效果如何？在 $\pi_{0.5}$ 上，RTC 的轨迹最平滑：肩关节的速度曲线没有折断、加速度峰值最低，执行速度比同步推理快 20%，六个双臂任务上任务吞吐的提升都统计显著 ($p < 0.05$)。

更说明问题的是延迟鲁棒性。注入 +100 ms ($d \approx 11$ 步) 和 +200 ms ($d \approx 16$ 步) 额外延迟：RTC 的任务吞吐几乎不变，同步推理线性下降，Temporal Ensembling 在 +100 ms 时直接触发保护停机。在最精密的“划火柴”任务上（这个任务没有重试机会），大延迟下 RTC 仍保持高成功率，同步推理完全失败。

代价是推理本身慢约 28% (97 ms 对 76 ms)，因为 VJP 要多做一次反向传播。和竞品 BID (Bidirectional Decoding) 比，BID 用 rejection sampling 保证跨块连续性，每次采 32 个块，计算量是 RTC 的 2.3 倍，高延迟下表现还不如 RTC，而且它需要一个额外的弱策略 checkpoint。RTC 完全免训练，对任何 flow / diffusion VLA 开箱即用；diffusion policy 也能在推理时转成 flow 之后套用。

它没解决的那一半。RTC 管的是“一块怎么平滑地接到下一块”，**反应延迟**它没碰。冻结区那 d 步是锁死的，即使这期间观测到了新情况（物体突然移动、抓取失败），这 d 步的动作也改不了。对高动态、要求即时反应的任务，瓶颈落在**从观测到第一个可修改动作的时间**（TTFA, Time to First Action）上，轨迹平不平滑是次要的。下一节 FASTER 冲的就是这个数。

1.4 FASTER：不只不卡，还要反应快

解决什么问题？ 执行平滑了还不够。FASTER[2] 对 RTC 提了一个很尖锐的批评：**平滑不等于快**。动作连续，不代表机器人能对突发变化及时响应。机器人正执行一个动作块时环境突然变了（球飞过来了），它多快能反应，取决于两件事：发出新推理请求后要等多久才拿到**第一个动作**，以及下一次推理什么时候触发。RTC 缩短了推理间隔，没有缩短第一个数。

论文把反应时间算清楚了，结论是它是一个**均匀分布的随机变量**，而非常数：

$$\Delta t_{\text{react}} \sim \mathcal{U}(\Delta t_{\text{infer}}, \Delta t_{\text{infer}} + s \cdot \Delta t_{\text{ctrl}})$$

Δt_{infer} 是推理延迟， s 是执行视界（每次推理后实际执行几步，常见 4 到 10）， Δt_{ctrl} 是控制周期（30 Hz 时 33 ms）。事件可能在执行窗口的任何位置发生，所以反应时间在这个区间上均匀分布，期望是 $\Delta t_{\text{infer}} + \frac{s}{2} \cdot \Delta t_{\text{ctrl}}$ 。

这个式子给出一个反直觉的结论：**从同步推理切到异步推理，期望反应时间只减少了 $0.5 \Delta t_{\text{infer}}$** 。异步的收益远比想象的小，真正的瓶颈是 Δt_{infer} 本身，而它里面最大的一块是“推理时要把所有去噪步跑完”。

Flow VLA ($\pi_{0.5}$ 、X-VLA 这些) 从高斯噪声出发做 N 步 Euler 积分去噪， N 典型取 10。传统做法给块内**所有动作**用同一套等步长调度：第 0 步动作和第 49 步动作都要等满 10 步，才能一起发出第一个动作。于是

$$\text{TTFA} \approx \Delta t_{\text{VLM}} + N \cdot \Delta t_{\text{AE}}$$

其中 Δt_{VLM} 是 VLM 骨干一次前向的时间， Δt_{AE} 是动作专家（Action Expert）单步去噪的时间。 N 步动作专家就是延迟的主体。这就是“恒定调度等于反应延迟瓶颈”的含义。打乒乓球这种任务，球从对方拍上弹出到必须挥拍只有几百毫秒；同步推理根本来不及，异步推理消除了块间停顿、但看到新球位后仍要等满 10 步去噪，球已经落地了。

怎么做的？ FASTER 的关键观察是：近端动作路径直，**一步就能估准**，只有远端动作才真正需要多步去噪。三条理由叠在一起：近端动作受当前观测强约束，解空间很窄；有了 RTC 那套动作前缀条件，近端动作还额外受已执行动作的约束；远端动作要预测未来，不确定性才真的大。论文没停在直觉上，用**路径直度（straightness）**这个指标量了一遍：块里前 1 到 10 帧的流动路径明显比后面的帧更直，路径越直，少步插值的误差就越小。

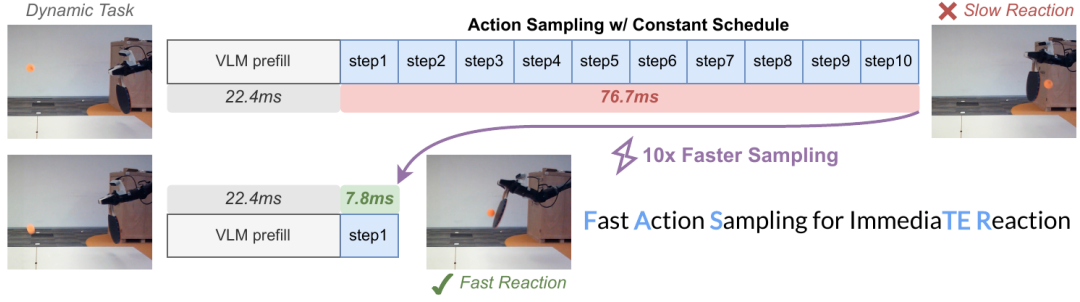


图 3: 上下两条时间线都从 22.4 ms 的 VLM prefill 开始：恒定调度要跑满 10 步去噪、再花 76.7 ms 才出第一个动作；FASTER 一步出近端动作，这一段压到 7.8 ms（图片出自参考文献 2）

于是让近端动作一算好就立刻发给机器人，不必等整块凑齐。

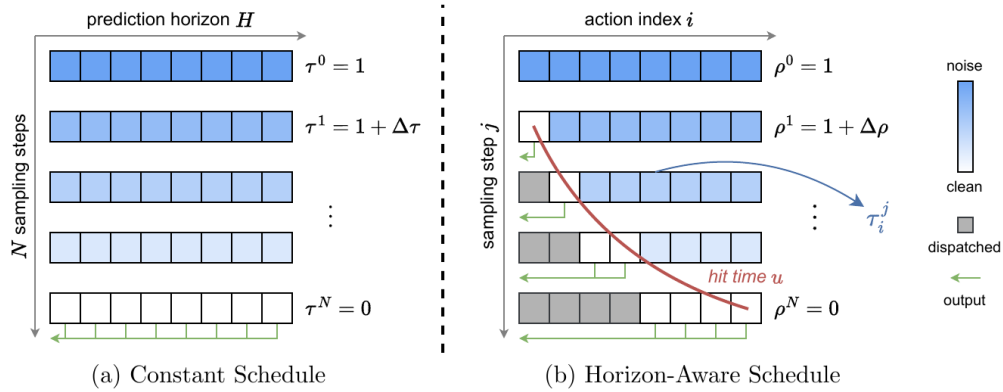


图 4: 左右两块的每一行是一个采样步、每一格是一个动作。左边恒定调度整行一起变干净，要到最后一行才能一次性发出；右边 Horizon-Aware 调度让左侧近端动作先变干净（图中的灰格即“已派发”），一算好就沿绿色箭头发出去（图片出自参考文献 2）

机制上，FASTER 给块内每个位置一个自己的 timestep，把原本一个标量的去噪进度换成一个向量 $\tau = \{\tau_i\}$, $i \in [0, H-1]$ 是动作在块内的位置。每个位置有自己的命中时刻 (hit time) u_i ，全局采样进度 ρ^j 从 1 降到 0 的途中，一旦降到 u_i ，位置 i 的动作就算去噪完成：

$$u_i = \left(1 - \left(\frac{i}{H-1}\right)^\alpha\right) \cdot u_0, \quad \tau_i^j = \max\left(0, \frac{\rho^j - u_i}{1 - u_i}\right)$$

u_0 是第一个动作的命中时刻，取 $(N-1)/N$ ，于是它一步就完成； $\alpha \in (0, 1]$ 控制命中时刻沿块的分布形状， $\alpha < 1$ 时近端动作完成得更早。 $\rho^j \leq u_i$ 之后 τ_i^j 恒为 0，那个位置不再被更新。

动作因此变成流式产出，FASTER 配了一套流式客户端-服务端流水线接住它：服务端每完成一批动作就立刻发给客户端、并发继续精化剩下的；客户端持续监听，收到就填进动作缓冲区，缓冲区不空机器人就一直执行。缓冲区满时还能触发提前停止，把服务端剩下的去噪步直接跳掉，推理周期随之缩短、推理频率提高，反应时间的上界又被压低一截。

这套调度和 RTC 天然兼容：RTC 把已执行动作当前缀条件喂进去，本质上就是给近端动作施加更强的约束（timestep 趋近 0），与 FASTER 的设计同向，两者叠起来更好。

备注：调度本身是推理期的，但直接拿它去跑一个预训练模型会遇到**调度分布偏移**——模型训练时从没见过随位置变化的 timestep。FASTER 的解法是**混合调度微调**：以概率 p 用新调度、以概率 $1-p$ 保留原来的恒定调度，让模型同时学会两种 timestep 参数化。架构一个字不改，能直接嵌进现有微调流程。所以“即插即用”这句话要带一个限定：调度是免训练的，用好它要一次轻量微调。

效果如何？相对 $\pi_{0.5}$ 与 X-VLA，即时反应动作的去噪从 10 步压到 1 步，论文报的是 **10× TTFA 加速**；乒乓球实机验证里，基线因为反应延迟失败、FASTER 能稳定回球；RTX 4060 和 RTX 4090 上都有效，不需要服务器级 GPU。

读这个“10×”要留神它量的是哪一段：论文压缩的是**近端动作的去噪采样**，从 10 步 76.7 ms 变成 1 步 7.8 ms，约 10 倍；而首动作延迟前面还挂着一段 22.4 ms 的 VLM prefill，谁也压不掉，所以端到端的首动作延迟是从约 99 ms 降到约 30 ms，三倍出头。“**采样快 10 倍**”和“**反应快 10 倍**”是两件事，这类加速比读的时候先问一句“分母里含不含 prefill”。

RTC 和 FASTER 都属算法层：只改推理期的调度策略，不动模型、不重训（FASTER 那次轻量微调不改架构）。两者把这一层的空间基本榨干了，剩下的瓶颈落在模型单次前向本身。

1.5 Realtime-VLA V1：把模型本身榨到 30 FPS

解决什么问题？上面算法层那些做法，都在“单次推理耗时给定”的前提下把停顿和抖动藏起来；它们没有让模型本身变快。可现实是以 PaliGemma 3B 为骨干的 π_0 ，官方 openpi/jax 实现双视角要 53.7 ms，照着模型结构直写的 PyTorch 更要 106.5 ms，而 30 FPS 实时控制的门槛是 ≤ 33 ms。低于这条线，摄像头视频流就必须丢帧，而关键事件一旦恰好落在被丢掉的那一帧上，反应延迟就凭空多出整整一帧。

这里的“实时”是严格的控制意义：**每一帧都能被处理，看的是最坏情况延迟 ≤ 33 ms，不是平均帧率 ≥ 30** 。这一节换一个层面解题：不改算法、不改训练，纯靠工程手段把模型本身在单张消费级 GPU 上跑进 33 ms。

这条线的第一篇论文题为《Running VLAs at Real-time Speed》[3]，后续两篇顺着它自称 V2、FLASH，所以下文按 **Realtime-VLA V1 / V2 / FLASH** 这个通行叫法称呼三者。

先要弄清时间花在哪。三类开销独立叠加：

- **Python/CPU 开销**。 π_0 单次前向要启动**一千多个 CUDA kernel**，每次启动都要 CPU 介入（Python 解析、kernel dispatch、指针传递）。单次只有几微秒，一千多次叠加、再加上 CPU 与 GPU 的同步等待，累计能占到总推理时间的**约一半**。
- **计算图冗余**。原始实现忠实按模型定义跑图，可很多操作等价可合：RMSNORM 的仿射参数本质是线性变换，能直接吸收进下一个线性层的权重；QKV 是三个独立矩阵乘，能合成一个大矩阵乘

再切片；动作时间嵌入的两段线性层之间没有非线性，能折成一段。这类冗余在学术代码里很常见，每个模块单看都正确，整体跑起来却有大量不必要的 kernel 切换和显存读写。

- **多视角把前两类都放大。**视觉编码器（SigLIP，400M 参数）对每个视角独立处理，视角越多计算量线性增加，计算图也越长。

怎么做的？ V1 的做法是分层逐一消除，先摘低垂的果子，再深入 kernel 细节。

第一层用 CUDA Graph 消掉 CPU 开销。把整次推理的 kernel 序列“录”下来，之后每次推理直接回放，由 GPU 和驱动负责调度，Python 与 CPU 完全退出关键路径。它的前提是每次推理的 kernel 序列和缓冲区指针不变，而 transformer 块没有动态分支，天然满足。这一步就省掉约 **50%** 的推理时间（从 106.5 ms 降到 50 ms 量级），是整个流程里增益最大的单步。

第二层做计算图的等价简化。RMSNorm 的仿射参数预乘进后续权重矩阵；动作时间嵌入折成一次变换，而且 timestep 只有 10 种取值，索性预计算成查找表、退化成加一个偏置；QKV 三个投影合并成一个大矩阵乘，RoPE 旋转位置编码也融进矩阵乘并预计算权重。三个变换合计省约 **7 到 8 ms**，同时把 kernel 数量降下来，为第三层打基础。

第三层是逐个 kernel 手调。经过前两层，整张网络剩下约 **24 个 GEMM 类算子**加配套标量算子，分布在三个模块里，逐一用 Triton 优化：

模块	roofline 下界	优化后实际	瓶颈类型	主要手段
视觉编码器 (SigLIP)	约 2.5 ms	约 4.1 ms	计算密集	手调 tile 参数、门控线性层融合
语言模型 (Gemma, 2.6B)	约 10.7 ms	约 12.5 ms	计算密集	同上，GELU 门控融合省约 1.7 ms
动作专家 (300M)	约 6.5 ms	约 11.0 ms	带宽密集	Partial Split-k、标量算子融合

这张表最值得记的是**两类瓶颈要用两套策略**：视觉编码器和语言模型是计算密集的，瓶颈在算力 (tensor core 利用率)；动作专家矩阵形状小、标量算子相对开销大，是带宽密集的，瓶颈在显存读写速度。门控线性层融合针对的是后者：FFN 里两个权重矩阵并行算、共享同一份输入 tile 的加载，结果合并后只写一次。动作专家那个 $512 \times 1152 \times 1152$ 的特殊 GEMM 用 Partial Split-k，拆成两个子 GEMM 分别对齐 SM 数量 (RTX 4090 有 128 个 SM)，再合并写入单一 kernel。

优化到什么程度算够？论文用 roofline 模型（显存带宽加 tensor core 算力）给 24 个 GEMM 建了理论下界，加上 CUDA Graph 下约 1.72 ms 的 kernel 同步开销（用 software barrier 可降到 0.86 ms），双视角推理的理论下界约 **20.6 ms**，而实现是 **27.3 ms**，剩余空间 $\leq 30\%$ 。这个对比说明当前实现已经贴近理论极限。

跑进 30 Hz 只是起点。V1 进一步提出 **Full Streaming Inference** 框架，利用 VLM（计算密集）和动作专家（带宽密集）的资源互补性让两者**并发跑**，把系统控制频率拆成三层。

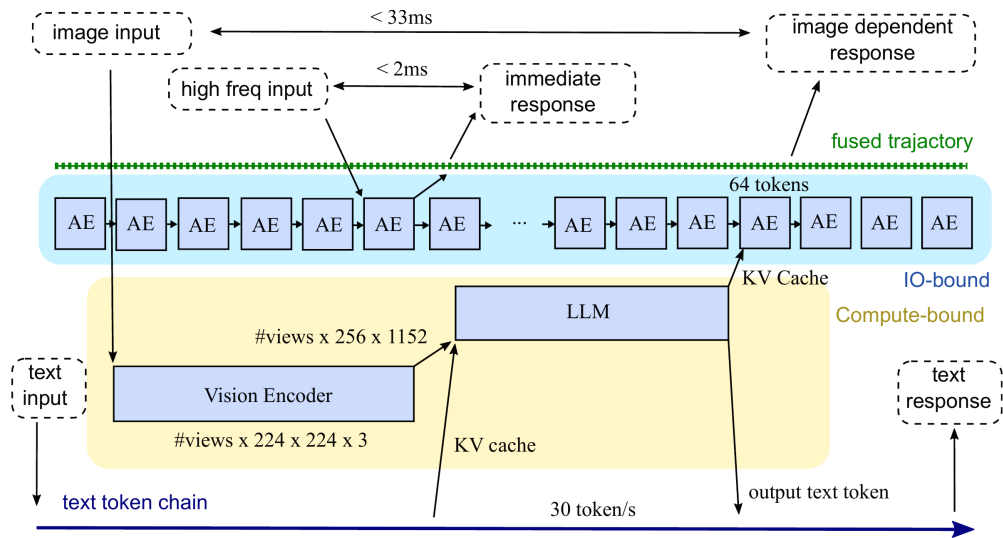


图 5: Full Streaming Inference: 大模型慢回路看场景、Action Expert 快回路出轨迹，并发跑出三层不同频率（图片出自参考文献 3）

控制层	输入信号	频率	角色
文本流 (VLM 捎带)	语言指令 / 思维链推理	< 1 Hz	高层任务规划、用户交互
视觉感知 (VLM)	多视角 RGB	30 Hz	场景理解，更新 KV cache
动作执行 (动作专家)	VLM 的 KV cache + 高频传感器	最高 480 Hz	实时轨迹生成、力控反应

两条回路各管一头。快回路由动作专家主导：高频传感信号（力传感器采样率可达 2 kHz 以上）注入它的输入 token，一次前向约 2 ms，更新轨迹缓冲区，适合紧急停止和力控调整。慢回路由 VLM 主导：新图像帧进来，VLM 前向 1/30 秒，更新 KV cache，动作专家读到新的 KV cache 后行为随之改变，适合响应场景变化。两条回路在同一张 GPU 上并发执行，资源互补，几乎不互相阻塞。

效果如何？单张 RTX 4090、BF16、无文本 prompt、chunk 长度 63：

视角数	朴素 PyTorch	openpi/jax	V1 优化后
单视角	105.0 ms	43.8 ms	20.0 ms
双视角	106.5 ms	53.7 ms	27.3 ms
三视角	113.9 ms	67.6 ms	36.8 ms

双视角跨过了 30 FPS 那道 33 ms 的门槛，论文给出的部署口径正是 30 Hz。

真机那个验证很值得看：两个竖排夹爪，上夹爪松手让笔自由下落约 30 cm，下夹爪要在笔落到位时夹住。输入是 30 FPS 的 USB 摄像头（本身约有 2 帧延迟），由 π_0 控制下夹爪开合。10 次测试 **10 次都抓住**，端到端反应时间 < 200 ms，与人类平均水平相当。这个结果的意义不在“学到了什么”（这个任务 LeNet 也能学会），而在于它从系统角度证明了：在十亿参数量级的大模型框架内，做到类人反应速

度是可行的。

关于那个 480 Hz 要说清一件事。论文的口径是**控制频率**意义上的 480 Hz：动作专家约每 2 ms 接收一次新传感信号并输出更新轨迹，这与“轨迹节点密度可以靠插值凑出来”的轨迹频率不同。不过它仍**不等于力矩环或阻抗控制环的频率**：能不能真正满足力控 / 阻抗控制的要求，还取决于力传感、总线周期、实时操作系统、控制器稳定性与抖动，需要由底层实时控制栈单独验证。

V1 的 Triton kernel 优化后来成了 V2 和 FLASH 的底层实现基础。没有它把单次前向压进 27 ms，后面那些学习化方法和投机推理都无从谈起。

它没解决的那一半。V1 是纯工程优化，不改训练、不改算法，只让已有模型跑得更快。于是问题换了一副面孔：**能跑这么快，不等于该一直跑这么快**。480 Hz 运行的动作专家需要对力、触觉这类高频传感信号做出响应，可现有 π_0 的训练数据里根本没有这类信号。“快”和“稳、准”要同时训进模型，这是 V2 的事。

1.6 Realtime-VLA V2：学会何时快、何时慢

解决什么问题？V1 把模型跑快之后，最直接的用法是给动作块的执行速度乘一个固定的加速因子（比如 $2\times$ 或 $3\times$ ）。结果让人沮丧，三件事同时出现：轻量化机械臂（QDD 电机、低刚度）在剧烈加速时开始抖动、甚至失控；示教数据全是人类用“舒适速度”操作的，强行加速会把机器人推到训练分布之外的状态，失败率骤增；而且一个任务里高速没问题的阶段和必须精确的阶段是**交替出现**的，统一设一个因子根本行不通。

折衬衫就是个现成例子：手臂空程移到衬衫上方要尽量快，折袖子要慢下来、还得是特定速度才不会让布料滑动，最后把衬衫移到角落又该快。固定因子对整段轨迹一刀切，加多了精确阶段失败，加少了空程阶段白等。

这三件事还是**连锁**的，不是并列的三条：抖动会破坏视觉输入的稳定性，视觉一抖 VLA 的预测更不稳，更不稳的预测又生成更抖的命令。“**稳**”是“**快**”和“**准**”的前提。

所以问题的本质落在**何时快、何时慢**这一侧，“能不能快”已经不是问题。这是要从经验里学的知识，调工程参数调不出来。V2[4] 的回答是一套四层的全栈方案，每层对着一个明确的物理问题：时延标定 → 轨迹后处理 → 空域控制 → 学习化速度自适应。

怎么做的？第一层先把时间对齐。训练时模型看到的世界是理想化的：观测即时、动作立刻生效。真实部署里每个环节都有时延：

时延来源	测量值
相机实际曝光提前量	55 ms
图像从相机读出	33 ms
关节位置反馈延迟	50 ms
运动动力学滞后（PD 控制跟踪误差）	150 ms

四项加起来约 **288 ms**，最大的一块是运动动力学滞后。不标定的话，模型收到的是“旧图像加旧关节状态”却以为是当前时刻，训练与推理的时间对齐严重错位，后面学到的“什么时候该减速”全会错位。

标定办法很朴素：让机器人在相机前做正弦摆动，同时在屏幕上显示三条进度条，分别代表系统当前时间、关节命令位置、关节实际位置的相位；用高帧率相机录下来，从像素值做相位估计，把标定精度做到 5 ms。

补偿要分两类。相机曝光与读出这两项合并成一个偏移量，从历史关节位置缓冲区里取对齐的时刻，保证图像和关节状态进 VLA 时是同一时刻的。那 150 ms 的运动滞后**不能靠“把命令提前发”解决**：它是 PD 控制器的跟踪误差、本质是控制回路动态，不是固定的传输延迟。它要交给后面的预放大。

第二层做轨迹后处理，只动时间不动路径。后处理不改变路径形状，只在时间维度上重新分配每一步的时长。这一点很关键：形状不变意味着这层对模型是透明的，不会破坏 RTC 那套前缀匹配。速度模型（第四层）先给出每一步的速度缩放因子定下整体节奏，然后做一次时域优化，把可能出现的加速度峰值摊平到整段动作里。对每一步的时长 Δt_i 求解一个二次规划：

$$\min_{\Delta t_{1:H}} \lambda_0 \sum_{i=1}^H \|\Delta t_i^{-1} - (\Delta t_i^{\text{ref}})^{-1}\|^2 + \lambda_1 \|a_i\|^2$$

Δt_i^{ref} 是速度模型给的参考时长， a_i 是关节加速度。这个式子的直觉是“弹簧加阻尼”： λ_0 项把时长拉回参考值、保住速度目标， λ_1 项惩罚加速度、保住平滑。用 OSQP 求解，快到能塞进推理链路里实时跑。

第三层在机器人本地补运动滞后。时域优化给出的是“应该走的轨迹”，那 150 ms 还在。这一层做两件事。一是状态估计，用一个线性递推模型估计机器人现在真实在哪：

$$\mathbf{q}_{k+1} = a \mathbf{q}_k + (1 - a) \mathbf{y}_k, \quad a = \exp(-h/\tau)$$

$\mathbf{q}_k$ 是估计的关节位置， $\mathbf{y}_k$ 是发出去的命令位置， h 是伺服周期， τ 是标定得到的响应时间常数。这个式子抓住了“实际运动总是滞后于命令”这个主要动态，靠命令历史就能在没有最新状态反馈时估出当前状态。

二是**预放大**：给定当前估计状态和未来参考轨迹，求解一个 MPC 问题得到真正发给机器人的命令，目标函数里带命令幅度的过冲惩罚和命令变化率的平滑惩罚，防止过激预放大引发振荡，用 acados 的 SQP-RTI 模式实现。它的效果用一句话说：**发出去的命令比模型期望的路径幅度更大，命令先过冲，经 PD 控制器衰减之后机器人实际走的路径恰好和模型预期对齐。**

时域优化和空域控制是两层独立的保障，缺一不可：前者保证轨迹在时间维度上平滑，后者保证硬件真跟得上。时域优化不施加硬约束，空域控制才是最后一道防线。

第四层学“何时快何时慢”。前三层解决的是“给定速度目标怎么稳定执行”，速度目标本身从哪来？V2 最有意思的部分在这里。

动机来自一张失败时间戳分布图：折衬衫任务在 $2\times$ 和 $3\times$ 加速下，失败并不均匀分布在整段轨迹上，而是集中在极少数几个特定阶段（折袖子）；其余大部分时间高速执行完全没问题。这说明**一刀切的固定加速因子，是在用最关键阶段的速度上限限制整个任务。**

于是找一个最好的速度决策者：人。做法是机器人在 VLA 策略控制下执行任务，操作员手持一个“油门”输入设备实时观察，觉得安全就加速、觉得快要出错就减速；所有油门输入配上当前观测（图像加关节状态）记录成监督信号；失败那些轮次的标签直接丢掉，免得错误标签污染训练集。然后每天采一批、训一版速度回归模型、第二天以更高的基础速度继续采，日迭代。

为什么不直接用强化学习？两个实践困难。一是样本效率：真机上失败一次代价太大，RL 要大量交互才学得到有意义的策略。二是分布偏移：高速下某些状态从未被访问过（被更早的失败“掩盖”了），模型很难估这些状态的价值。人在环方案绕开了两者，因为人给的是**连续回归标签**而非二值的成功失败，回归模型更难过拟合，数据还能跨天累积。

要注意这一层学的是一个**轻量的速度调度器，不动 VLA 策略本身**：它吃当前观测、输出一个速度缩放因子，插进第二层的轨迹后处理里。VLA 主模型一次都不用重训。

效果如何？三个任务从易到难覆盖不同精度要求。折衬衫在绝大部分过程以 $2\times$ 到 $3\times$ 执行、在折袖子阶段自动降回 $1\times$ 。放入夹具（5 个 PCB 先放进 shield、再把 shield 放进夹具，对准精度要求 **0.2 mm**）是“精确优先”的极端场景，结论是**模型确实学会了在对准阶段自动减速**，在这么苛刻的精度要求下成功率仍然保持。抓取并锁扣（抓 4 个工件、放入新夹具、锁上顶盖）每一步都要高精度，属于触及当前 VLA 极限的任务，全栈方案在它上面也做到了接近人类速度。

“接近人类速度”这个说法要看清基准：它比的是**操作员用相同动作风格时的速度**，不是正常人类的最快速度。另外全栈缺一不可，单靠 V1 的系统优化强行加速，失败率会大幅上升；标定、后处理、速度学习补齐之后失败率才回到可接受范围。

论文还给了一个漂亮的上界分析，思路和 V1 的 roofline 同源：任意一段轨迹要么**运动受限**（硬件的速度、加速度、jerk 已到极限），要么**控制受限**（动作块可用的那一段太短，再快预测就失效）。折衬衫任务的速度-加速度-jerk 曲线显示大部分段已经处在运动受限状态，也就是说速度适配算法已经做到局部最优，再往上要么改硬件，要么提升 VLA 预测本身的稳健性。

它没解决的那一半。三条局限。速度学习依赖人工标注，每天采数据要专业操作员，没法完全自动化；四层叠起来工程复杂度很高，移植到新机器人平台的成本不低；而且 V2 专注“怎么更好地用模型的输出”，**推理算力本身一点没再优化**。最后这条就是 FLASH 的入口。

1.7 Realtime-VLA FLASH：用投机推理少算几次

解决什么问题？前面两节都在让**每一次**推理更快。FLASH[5] 换了个问题：大部分时间真的需要每次都跑完整推理吗？

先看清 π_0 一轮完整推理的三个串行阶段：图像编码 11.3 ms、VLM prefill 26.7 ms、动作去噪 20.0 ms，合计 **58.0 ms**。分块执行的做法是预测出一段动作块（比如 50 步），执行其中 12 步，然后触发重规

划，拿最新观测重跑一遍完整推理。

这 58 ms 在延迟敏感的任务里是真瓶颈。传送带分拣任务，皮带速度 15 m/min，58 ms 内皮带已经移动 **14.5 mm**，物体可能已经离开最优抓取窗口。而矛盾在于：大多数重规划轮次里观测变化很小，新动作块和上一轮高度相似，完整推理在反复做重复劳动。

roofline 分析指出了靶点：图像编码和 VLM prefill 是**计算密集**的，加速要靠减少计算；动作去噪是**带宽密集**的，每步去噪反复读 KV cache，计算资源大量闲置。**这个闲置正是可乘之机**：既然算力空着，不如同时跑多个验证点，而不是老老实实串行走 10 步。

怎么做的？ FLASH 把大模型那边成熟的**投机解码 (speculative decoding)** 移植到连续动作空间，做成双路架构：绝大多数轮次走便宜的快路径，验证失败或进入精细阶段时回退完整路径。三个组件。

组件一，一个约 110M 的草稿模型。结构是单个 Gemma block 加一个线性动作头，输入当前图像特征、语言指令、机器人状态，再加 H 个可学习的“动作查询”（每个对应块里的一步）；注意力掩码用 blockwise 设计，让这些查询能并行关注整个条件前缀，于**是一次前向就吐出完整动作块**。训练是对真实动作块做带位置权重的监督回归：

$$\mathcal{L}_{\text{draft}} = \sum_{h=1}^H w_h \ell(\hat{a}_{t+h-1}^d, a_{t+h-1})$$

w_h 是位置权重，近端步权重更高。110M 只占主干 VLM (2.7B) 的约 4%，生成成本极低。它**不做去噪、直接回归目标**，这是它快的根本原因，也是它会犯错的根本原因，所以必须有验证环节把关。

组件二，多步并行验证。连续动作空间没有 token 概率，怎么判断草稿对不对？flow matching 的结构给了钥匙：它训练时学的是“从高斯噪声到动作终点”这条线性插值路径上的速度场。给定草稿终点 $\hat{A}_t^{(d)}$ 和噪声 ϵ ，可以在这条线性路径上任取一个中间点

$$\tilde{A}_\tau = \tau \hat{A}_t^{(d)} + (1 - \tau) \epsilon$$

再用动作专家（复用上一轮的 KV cache，**跳过昂贵的 VLM prefill**）预测该点处的速度，反推终点估计：

$$\hat{A}_t(\tau) = \tilde{A}_\tau + (1 - \tau) v_\theta(\tilde{A}_\tau, \tau | c_t, s_t)$$

草稿要是和主模型一致，从不同 τ 反推出的终点估计都该和草稿 $\hat{A}_t^{(d)}$ 接近。选几个验证时间点 $\mathcal{T} = \{\tau_1, \dots, \tau_K\}$ **同时**做这个重建，用掉那些闲置算力，这就是前面说的“可乘之机”。

接受准则取的是**最长合法前缀**，粒度比全接受或全拒绝细：逐步检验动作块里每个动作在所有验证点下的距离是否低于阈值 δ ，取都通过的那个最长前缀 L 。这个粒度更细，快路径与完整路径之间的切换也更平滑。 $L = 0$ 时草稿被拒，立刻回退。

备注：投机推理为什么不损精度？因为验证保证了等价性。被采纳的动作必须通过动作专家的一致性检验，没通过的部分立刻切回完整推理，所以 FLASH 的输出在数学结构上等于“主模型认可的前缀 + 必要时的完整重算”，草稿模型没有独走的机会。

组件三，分阶段回退。验证是局部的：它检查当前草稿与主模型是否一致，却预见不到即将到来的精细调整阶段。以“把碗放进盘子”为例，平滑移动阶段快路径跑得很好，可一进入最终放置（碗接近盘沿），草稿模型的小误差会被放大、碗就落偏了。FLASH 的办法是把**夹爪开合翻转**当作“即将进入精细阶段”的信号：候选动作块里任何一个验证分支检测到夹爪状态翻转，立刻回退到完整路径重新生成。这是个启发式，但很有效，因为抓取和放置那一瞬恰好是整个任务里最精细、容错空间最小的时刻，也正是草稿误差最容易被放大的时刻。

于是回退有两个触发条件：验证失败（ $L = 0$ ，草稿与主模型不一致，刷新 KV cache 后完整推理），以及相位转换（检测到夹爪翻转，主动回退以保住精细阶段的质量）。

效果如何？在 LIBERO 基准上（配合 Triton 内核，RTX 4090D）：投机轮最快 **7.8 ms**，完整轮 58.0 ms，单轮加速约 7.4 倍；混合之后任务级平均延迟 **19.1 ms**，端到端加速 **3.04 倍**，动作吞吐提升 2.63 倍；四个 LIBERO suite 的成功率与 Torch- π_0 基线基本持平。真机那条更直接：传送带 15 m/min 的分拣任务里 FLASH 能成功抓取，完整推理基线失败。

备注：19.1 ms 是**混合平均**，不是每轮都能到 7.8 ms，实际比例取决于草稿命中率。LIBERO 上命中率高（机器人动作平滑、帧间变化小）是这个数字好看的主要原因；换到动作剧烈变化的场景，回退会更频繁、加速比会下降。

另外要留神这一节的 58 ms 和 1.5 节那个 27.3 ms 不是同一个口径：后者是双视角、BF16、无文本 prompt、chunk 长度 63，在 RTX 4090 上量的；前者是 RTX 4090D 上含图像编码的完整三阶段。**同一条技术线的两篇论文，分母都可能不一样**，这正是 1.4 节那条教训的又一个例子。

它没解决的那一半。验证阈值 δ 目前是固定的，理想情况下该做成沿轨迹自适应（任务早期宽松、精细阶段收紧）；夹爪翻转是个启发式信号，更好的做法是让模型自己学会预测“即将进入精细阶段”。FLASH 压的是重规划频率下的平均延迟，与 V1 的内核优化可以叠加，与 V2 的学习化速度调节理论上也可以组合。

1.8 Legato：把续接从推理时的补丁变成训练时学到的属性

解决什么问题？RTC 那套续接机制很漂亮，但它有一个被刻意接受的设定：**整套机制完全活在推理时，策略本身从没“学过”要延续上一块**。训练时学的还是标准 flow matching 速度场，续接只是测试时外挂的一层启发式补丁。Legato[6] 从这一点切入，主张稳定的分块执行要求“续接”成为策略**训练时就学到的原生属性**。

它和 RTC 用的是同一套异步执行结构（推理延迟 d 、执行步 s 、ramp 长度 r ，常满足 $r + s + d = H$ ），也同属算法层：不碰模型骨架、不靠 kernel 加速，只改动作块怎么生成、怎么衔接。

先补一个后面要反复用到的事实。每个周期只真正执行 s 步就切下一块，所以新块的“第 0 步”在真实时间上对应的正是上一块的“第 s 步”，相邻两块整整错开 s 。也就是说，新块开头那段覆盖的恰是上一块“还没来得及执行”的那段计划。把这两段对齐接上，就是续接要做的事。

把续接只放在推理时，会带来两个毛病：

- **训练与推理不一致。**策略训练时从纯噪声出发一步步去噪成动作块，中途没有任何“延续上一块”的干预；可推理时 RTC 在每一步去噪前都往参考动作上拽一把。**训练时没见过的事，推理时天天在做**，这种分布错位让续接行为不稳定，还对推理时那些启发式细节（怎么 clamp、引导多强）很敏感。
- **多模态乱跳。**flow 策略天生多模态，面对“用左手还是右手抓”“绕左边还是右边”这类有多个等价解的情形，它会在几个模式间反复横跳。策略本身没学过“要延续上一块的选择”，相邻两块就很容易选到不同模式，表现出来是犹豫、突兀变向，轨迹不本质平滑，任务完成时间被白白拖长。

怎么做的？动手之前 Legato 先回答一个看似技术、其实很关键的问题：续接的引导，在去噪过程里该施加几次？

一个自然的想法是只在**初始化时**把重叠区的噪声替换成参考动作，之后正常多步去噪，这样训练（标准 flow matching）和推理就对得上了。论文用实验否掉了这个想法：只在起点引导一次，**随着去噪步数增加，重叠区会逐渐漂移、偏离参考动作**。原因很直白，模型在重叠区学到的速度一般不为零，每步去噪都把这段往外推一点，累积下来就跑偏了。

于是得到一个结论和一个两难。结论是有效续接必须**每步引导**；两难是直接在标准 flow matching 上外挂每步引导，又回到“训练时没见过、推理时硬加”的不一致，这正是 RTC 的处境。Legato 的破局点：**与其在推理时外挂每步引导，不如在训练时重塑 flow 动力学，让模型原生就支持这种逐步引导。**

第一步，把续接强度写成一条 schedule。用一个横跨整个动作块的向量 $\omega \in [0, 1]^H$ 描述每个时间步要多严格地延续上一块：开头 d 步 $\omega = 1$ 满引导（对应推理延迟区，必须一致），随后长度 r 的 ramp 段从 1 平滑降到 0，尾段 $\omega = 0$ 完全自由生成。整条 schedule 由两个标量 (d, r) 唯一确定。和硬 clamp 前缀相比，这个**软 schedule** 的 ramp 段给出了对续接强度与平滑度的精细可调控制。

第二步，把“部分已知”塞进去噪的起点。标准 flow matching 从纯噪声起步，Legato 换成动作与噪声的逐位混合：

$$\epsilon_{\text{eff}} = \omega \odot \mathbf{A} + (\mathbf{1} - \omega) \odot \epsilon$$

训练路径就是从这个混合起点连到真值： $\mathbf{Y}_t = (1 - t) \epsilon_{\text{eff}} + t \mathbf{A}$ 。直觉是 $\omega = 1$ 的位置起点就是动作本身（已知，不用去噪）， $\omega = 0$ 的位置起点是纯噪声（退回标准 flow matching）。

这里的 $\mathbf{A}$ 在训练和推理是**两个不同的东西**。训练时有真值， $\mathbf{A}$ 直接就是数据里的真值动作块；推理时没有真值，得用上一块的预测临时拼一个参考 $\mathbf{A}_{\text{ref}}$ 出来。怎么拼，用到前面那个错位：

- **截断。**上一块预测的 H 步里，前 s 步已经执行、成了过去，丢掉；只留还没执行的后 $H - s$ 步，平移到新块的本地坐标上当参考。留下的长度恰好 $H - s = d + r$ （由 $r + s + d = H$ 保证），正好

盖住新块里 $\omega > 0$ 、需要被引导的那一段。“有参考可依的位置”与“该被引导的位置”严丝合缝，这不是巧合。

- **补齐**。截出来只有 $d + r$ 个，凑够 H 还差 s 个，就拿最后一个动作值重复补上。这 s 个补出来的值正好落在尾部 $\omega = 0$ 的自由区，而 $\omega = 0$ 时参考在混合式里权重为零、根本不参与，所以**补什么都不影响结果**，补齐纯粹是凑张量形状。

用一组具体的数走一遍（ $H = 12$ 、 $s = 6$ 、 $d = 2$ 、 $r = 4$ ）：

步骤	内容
上一块预测	$\mathbf{A}_{\text{prev}} = [a_0, a_1, \dots, a_{11}]$
已执行 $s = 6$ 步	$a_0 \dots a_5$ 成了过去，丢弃
截断后	$[a_6, a_7, a_8, a_9, a_{10}, a_{11}]$ ，长度 $6 = d + r$
补齐 $s = 6$ 个	$\mathbf{A}_{\text{ref}} = [a_6, \dots, a_{11}, a_{11} \times 6]$ ，长度 12
对齐的 ω	$[1, 1, 0.75, 0.5, 0.25, 0, 0, 0, 0, 0, 0, 0]$
真正起作用的	前 6 位（ $\omega > 0$ ）；后 6 位是补的，被无视

读法：新块开头 2 步被钉死接上 a_6, a_7 （上一块紧邻的未来），中间 4 步平滑松手，最后 6 步完全自由生成。**机器人从旧计划切到新计划的那一刻，接缝两侧说的是同一件事**，于是轨迹连续、不跳变。

备注：两个易混点。 $\mathbf{A}_{\text{ref}}$ 取自上一块的**预测（计划）**，不是机器人实际走过的轨迹，续接对齐的是“新计划”与“旧计划的未竟部分”。另外它在这一块的 N 步去噪里**只拼一次、反复复用**：每步去噪前都拿同一个 $\mathbf{A}_{\text{ref}}$ 引导，“每步引导”指的是每步都引导一次，不是每步换一个参考。

第三步，重塑速度场。这是 Legato 的核心。推理时每步去噪前先把当前状态往参考动作拽、再去噪一步，把这个“引导-去噪”递推取连续时间极限，可以得到它**精确**对应的常微分方程（不是近似）：

$$\dot{\mathbf{Y}}(t) = (\mathbf{1} - \omega) \odot f_{\theta}(\mathbf{Y}, t) - \kappa \odot (\mathbf{Y} - \mathbf{A}), \quad \kappa = \omega / \Delta t$$

要让“推理时实际执行的速度场”恰好等于标准 flow matching 的目标，把上式反解出 f_{θ} 、代回路径，得到一个干净的闭式训练目标：

$$\mathbf{v}_{\text{target}} = (\mathbf{1} - \kappa \odot (1 - t)) \odot (\mathbf{A} - \epsilon)$$

$\kappa = \omega / \Delta t$ 是由 schedule 决定的引导刚度， $(\mathbf{A} - \epsilon)$ 是标准 flow matching 的速度方向。这条式子说了一件很优雅的事：**Legato 保留了标准 flow matching 的几何方向，只按 schedule 重塑了速度的幅值**。训练时网络就回归这个 $\mathbf{v}_{\text{target}}$ ，于是它学到的动力学与推理时逐步引导实际跑的动力学严格一致，前面那个两难被彻底解开。

顺着这个式子还能看出它为什么更平滑、更少乱跳：速度幅值被 ω 调制， ω 大的位置（重叠区加 ramp 段）速度被压小，而速度小就意味着这些位置**更不易变**，去噪过程中很难再在多个动作模式间乱切，多

模态乱跳被天然抑制；同时 ω 沿块从 1 平滑降到 0，续接从“严格延续”平滑过渡到“自由生成”，块边界不再硬接。

备注：把 ramp 长度 r 设为 0 时，Legato 的 schedule 退化成硬 overlap 约束，形式上接近“训练时的 RTC”。差别在于后者只是把续接当**外部硬约束**贴上去、不动底层 flow 动力学，Legato 是**重塑动力学**让续接成为策略的原生行为。论文实验里 Legato 全面优于训练时 RTC，而且非零 ramp 带来更平滑的过渡。

第四步，让它能真正部署。真实部署里推理延迟 d 会随硬件、模型大小、推理优化而变，ramp 长度 r 也可能想调来控制平滑度。schedule 要是写死，换个延迟就得重训。Legato 的办法是训练时**随机化** (d, r) 让策略见过各种 schedule，同时把当前 schedule ω 作为**条件**喂给模型（沿特征维拼到带噪动作上，形状从 (H, D_a) 变成 $(H, D_a + 1)$ ，这一行叫 condition row）。于是推理时只要改 ω ，**同一个模型就能适配不同延迟和平滑度，不用重训**。消融实验显示去掉 condition row，平滑度和执行稳定性都会下降。

效果如何？对照设置很干净：5 个真实操作任务（堆碗、倒入碗里、装箱、叠毛巾、开抽屉），统一 120 s 截止；Legato 与 RTC 从**同一个 $\pi_{0.5}$ 预训练 checkpoint** 出发，同数据、同超参、同步数，唯一变量就是续接方式。

结果是 Legato 在全部 5 个任务上一致优于 RTC，**轨迹平滑度和任务完成时间约各提升 10%**，任务完成分数也更高，更平滑、更快，不以成功率为代价。平滑度细看：衡量频域速度平滑的 NSPARC、衡量边界局部连续的 chunk-overlap RMSE 都显著改善；衡量全程 jerk 的 NLDLJ 不显著、但也不退化，因为它主要由重叠区之外的运动段主导，而 Legato 改善的是边界。最见效的场景是堆碗，多个外观相似碗带来大量等价模式，RTC 反复横跳，Legato 选定模式后保持一致、完成更快。泛化性也测了：换 schedule（变 s 、变 d ）、换 VLA 模型（ π_0 、GR00T N1.6、Kinetix 上的 UNet 策略）、换去噪步数，Legato 都稳定优于 RTC，说明它学到的续接行为不绑定某个特定骨架。

它没解决的那一半。Legato 当前把去噪步数 N 在训练时定死，速度场是在固定 N 下学的，推理时难以自由调整去噪步数（论文给了一个启发式 rescale 来缓解）。怎么让 N 也可变、同时保持训练推理一致，是它留下的开放问题。

1.9 本节小结

实时是三条线合力的结果，单点 trick 撑不起来。算法层管调度，消除停顿与抖动、压缩反应盲区；系统层靠榨硬件让模型本身进 30 ms，是上面两层的速度地基；学习加投机层在“已经快”的基础上把平均延迟压到极限。缺哪一条，都只是部分答案。

六篇工作的取舍摆在一起看，有两条能带走的判据。

第一条：加速比先找分母。这一节出现过三个“倍数”，量的都不是同一段。FASTER 的 $10\times$ 量的是近端动作的去噪采样（76.7 ms 到 7.8 ms），端到端首动作延迟因为前面挂着 22.4 ms 的 prefill，只有三倍出头。V1 的 106.5 ms 到 27.3 ms 是双视角、BF16、无文本 prompt、chunk 63 的口径。FLASH 的 58 ms 到 19.1 ms 是**混合平均**，其中 7.8 ms 那档只在草稿命中时才拿得到。三个数字都真实，可它们互相之间不能直接比大小。

第二条：接缝在哪，代价就在哪。六篇都在链路上切了一刀，每一刀都换来一段“没人能看见全局”的区间。RTC 冻结的那 d 步换来了平滑，代价是这 d 步里的新情况一律无法响应；FASTER 让近端动作一步出，代价是要多训一次混合调度、还要承担少步估计的误差；V1 把两条回路解耦，代价是快回路看不到最新的场景理解；V2 学到“何时该慢”，代价是这套知识来自人工标注、换个任务要重采；FLASH 让草稿先猜，代价是命中率一掉、加速比跟着掉；Legato 把续接学进策略，代价是去噪步数在训练时被定死。

这两条判据在后面三节还会反复用到。

2 长时记忆：让 VLA 记得住、想得到

2.1 单帧 VLA 的马尔可夫假设为什么不够

绝大多数 VLA (OpenVLA、 π_0 、ACT.....) 都有一个隐含假设：**当前这一帧，足以决定下一步动作**——也就是把单帧观测当成了充分统计量。

可真实操作任务常常是**部分可观测**的：单帧不足以恢复出决策所需的状态。

- 机械臂挡住了关键区域，当前帧看不全；
- “按按钮”任务，按前按后画面几乎一样，分不清按过没有（两个不同的真实状态映射到几乎相同的观测，这叫**状态别名**）；
- 多阶段任务里，早一步的决定影响晚一步该怎么走；
- 传送带上的物体在动，得预判它会移到哪。

要说准确的话：**环境的状态转移本身可以仍是马尔可夫的**，问题出在策略只拿到了部分观测——这类问题的标准刻画是 POMDP（部分可观测马尔可夫决策过程）。所以补救办法是把决策依据从单帧扩成足以近似状态的东西（马尔可夫性本身不用放弃）：引入历史、记忆，或做状态估计。

一句话：**只看当下一帧的 VLA，既不会用过去，也不会预判未来，于是长程任务做不下来。**这条研究线就是在回答——怎么给 VLA 装上“记忆”和“想象”。

2.2 一条主线：从隐式上下文到记忆 + 想象

这个方向的工作可以排成一条由隐到显、再从记忆走向想象的主线：

阶段	代表工作	一句话定位
看过去 1: 隐式	Long-Context DP	把历史当长上下文，用“也预测过去”正则 + 缓存训练，又好又省
看过去 2: 轻量	HAMLET	给现成 VLA 外挂可学习 token + 轻记忆，免重训就会用历史
看过去 3: 检索	Chameleon	观测-动作延迟；记忆要可区分 / 可寻址 / 可前瞻
看过去 4: 记忆库	MemoryVLA	仿海马体的显式感知-认知记忆库：检索-融合-巩固
看过去 5: 双记忆	Dual-Memory VLA	把“先验”和“近期动作一致性”也当记忆，生成又快又稳
看未来 1: 想象	MemoryVLA++	在记忆之上加世界模型，潜空间想象未来
看未来 2: 预测	BagelVLA	语言规划 + 视觉预测 + 动作交错统一，单步残差低延迟
长程收口	Long-VLA	输入掩码把子任务切成移动 / 交互两段，端到端拿到技能串接

前五行在“用好过去”，第六七行加上“预判未来”，最后一行直指“长程”。2.3 到 2.10 按这个顺序逐篇看，读的时候留意一件事：**记忆越做越显式，代价也越来越硬**，这一节末尾会把这笔账算清。

2.3 Long-Context DP：先问“能不能就把历史喂进去”

解决什么问题？ Long-Context DP[7] 是这条线上最轻的一种做法：不建任何记忆结构，直接问能不能把历史当成一段长上下文喂进策略。回答的过程把“朴素拉长上下文”的两个死穴讲透了。

- **学歪**。历史越长，画面里能和动作“碰巧对上”的特征就越多。策略一旦抓住这些虚假相关，部署时就偏离专家行为，性能反而下降。
- **算不动**。高维图像序列的显存与计算开销随上下文长度线性甚至更快增长，端到端训练长上下文策略贵到不现实。

正因如此，已有方法大多**回避**：要么截断上下文，要么抽几个关键帧把历史压成摘要。省是省了，但可能把后续决策真正需要的信息一起丢了。

论文还给了一个反直觉的观察。模仿学习里有个经典毛病叫 **copycat**：策略过度依赖上一步动作、照抄历史而不看观测。而扩散策略恰好是**相反的毛病**：它生成的动作序列，过去与未来之间的时序依赖**比专家数据还弱**。也就是说它没有抄过去抄太多，而是**根本没把过去用起来**。这个观察指出了一个比算力更本质的问题：光把历史喂进去不够，还得有办法逼模型真的去依赖历史。

怎么做的？三招，一招治“学歪”、一招治“算不动”、一招用在推理时。

过去-token 预测 (PTP)。常规模仿学习预测未来动作，PTP 加一个辅助任务：让策略在预测未来动作的同时，**也把过去的动作 token 预测出来**。给定历史观测 $o_{t-k:t}$ ，策略联合预测从过去 $t-k$ 到未来 $t+h$ 的动作：

$$\hat{a}_{t-k:t+h} = \pi_{\theta}(o_{t-k:t})$$

直觉是要能把“过去做过的动作”重建出来，模型就**必须在内部表征里保留历史信息**。这等于用一个自监督正则，把“依赖过去”硬塞进训练目标。

一个关键发现顺带落地了第二招：PTP 的收益主要来自**策略头**的序列建模，几乎不依赖视觉编码器。既然时序建模主要发生在策略头，那就把两者解耦，分三阶段训练：先用**短**观测上下文加长预测视界训练视觉编码器；然后冻结编码器，把训练集所有帧的 embedding 预计算并缓存；最后在缓存好的长上下文 embedding 上训练策略头。这样长上下文策略的训练开销近似短上下文水平，**训练加速 10 倍以上**。

测试时自验证。策略既然会预测过去动作，就能拿它给自己打分：同一观测下采多条候选动作序列，选与**刚刚已执行的过去动作最一致**的那条。

效果如何？6 个仿真加 4 个真实任务上验证（基于扩散策略）：性能 3 倍、训练加速 10 倍以上，成功率推到约 80%，而朴素长上下文基线远低于此。收益主要来自 PTP 对策略头的正则，而非更强的视觉特征，这也正是“缓存 embedding”能省下大量训练成本的根本原因。

它没解决的那一半。历史仍然是“一段被一起喂进去的窗口”，没有可检索、可挑选、可长期保留的结构。后面几节就是把记忆做得越来越显式。

2.4 HAMLET：给现成 VLA 外挂一个轻记忆

解决什么问题？现在已经有一大批预训练好的强 VLA（OpenVLA、 π_0 、CogACT、GR00T N1.5），它们几乎都是**单帧**的，每一步只看当前一帧做决策。要给它们补上历史感知，硬约束是：能不能不从头重训这些大模型？HAMLET[8] 回答的正是这个。

单帧的毛病很具体：“要不要移动手臂去放置”取决于**物体是不是已经被抓起来了**，物体一被遮挡，只看当前帧根本判断不了。那直接把过去几帧塞进输入行不行？作者实测**朴素追加 4 帧历史观测，前向就慢约 35%**，显存还膨胀、把可用 batch size 压小。对本来就很大的 VLA 来说，这种多帧重训代价高到不划算。

所以 HAMLET 的目标很明确：一个**即插即用、与骨干无关**的微调框架，不重训大模型、不需要额外大规模预训练。

怎么做的？标准 VLA 的数据流是 VLM 把当前观测 o_t 和指令 c 编码成隐表征 h_t ，再由动作专家从 h_t 预测动作块。HAMLET 要用历史信息增强 h_t ，只加两个轻量组件。

时刻 token。存原始帧不划算，又慢又占显存，而且画面里大量是静态冗余的背景。HAMLET 给每个时刻学一个紧凑摘要：在时间步 t 往 VLM 的输入序列里追加一组可学习的 token m_t ，和观测、指令一起过 VLM：

$$[h_t; m'_t] = \mathcal{F}_{\theta}([o_t, c; m_t])$$

VLM 是因果注意力，这组 token 会注意到当前观测和指令，于是输出的 m'_t 成了该时刻一个紧凑、任务相关的场景摘要。

怎么让它抓到关键线索而不是一堆雷同背景？用**时间对比学习**初始化：正样本取同一时刻的图像增强版（光度、模糊、噪声、遮挡扰动），训练目标鼓励不同时间步的 token **互相可区分**。这样它们自然会强调随任务推进而变化的动态部分、压抑静止背景。

记忆模块。有了每个时刻的摘要，还要跨时间步聚合成能喂给动作专家的时序条件。这里 HAMLET 强调一个观察：**并非每个时刻同等重要**。对所有历史时刻一视同仁地平均（论文里的 Moment Concat. 基线），冗余会淹没关键线索。所以它用一个轻量记忆模块**选择性**聚合，按重要性整合出一个历史感知的条件，再交给原来的动作专家。

整套东西只在原 VLA 外面加了一组可学习 token 和一个小记忆模块，骨干基本不动，因此可以当微调插件套到不同 VLA 上。

效果如何？长程最受益：在需要回看过去轨迹的长程真实任务上相对基线**提升约 47.2%**。跨骨干也成立：套到 GR00T N1.5 上在 RoboCasa Kitchen 达到 66.4%（基线 64.1%），套到 CogACT 上在 SimplerEnv-Bridge 达到 63.5%，通用基准 LIBERO 从 95.6% 提到 97% 以上。

它没解决的那一半。它聚合历史的方式仍然偏“汇总”，没有正面回答一个更尖锐的问题：**当下这一刻，到底该从历史里精确地找回哪一段？**那是下一节的事。

2.5 记忆要“找得回、用得上”：Chameleon 的三条标尺

Chameleon[9] 用**猜杯子**点破问题：先看到球藏在哪个杯子下、洗牌后球已不可见、最后却要选对——决定动作的信息早被看到、用时却已看不见，这叫**观测-动作延迟**。

这种“信息被观测到”与“信息被用于动作”之间的时间差，让操作在策略的观测空间里变成非马尔可夫的：**同样的当前画面，因为历史不同，要做不同的动作**。

作者强调一个关键判断：**这个问题不是“多给几帧”能解决的**，而且现有两类记忆做法都不够。紧凑摘要会把动作真正需要的细粒度事件轨迹抹掉，只记住“有个球被藏了”、却忘了“藏在哪个杯子、那个杯子怎么动的”；相似度检索则可能取回一个“画面看着像、但和当前控制无关”的历史帧。由此推出一个很犀利的结论：**具身记忆的好坏，不该由“存什么、取什么”来定义，而该由“是否让过去可用于当前控制”来定义**。

顺着这条定义，Chameleon 借人类情景记忆作功能类比（不是生物蓝图），提炼出面向控制的记忆必须满足三件事：

- **可区分**：知觉上相似的历史不能塌缩成同一个记忆状态，否则“球在左杯”和“球在右杯”两段相似画面会混成一团。
- **可寻址**：做决策时要能**按当前这个选择所需**去检索对应的那条历史轨迹，而不是泛泛地取个相似帧。

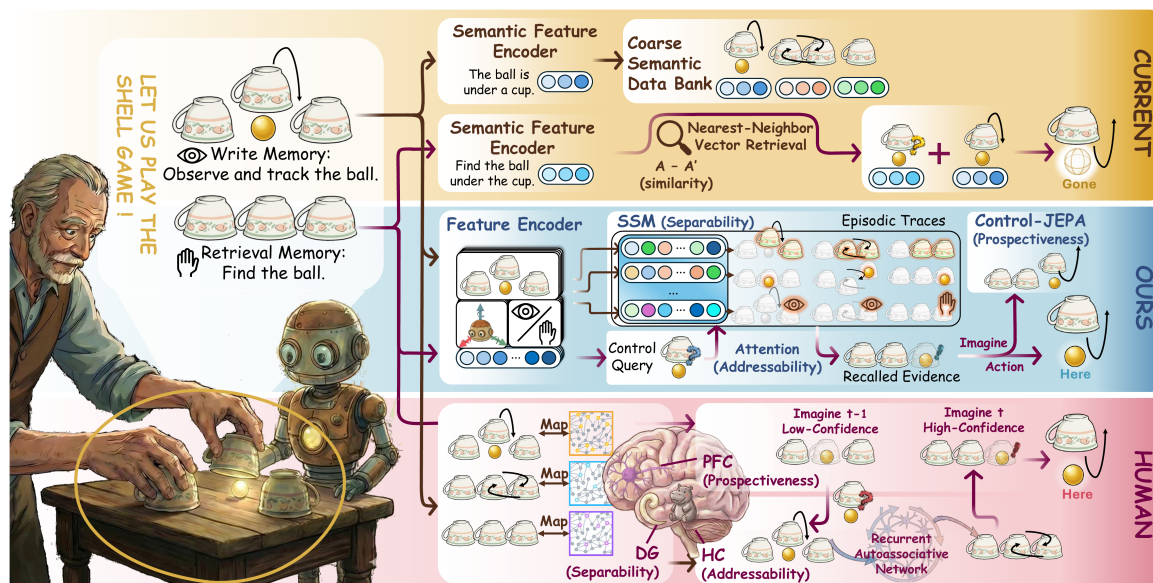


图 6: 观测-动作延迟需要控制索引的记忆：可区分 / 可寻址 / 可前瞻（图片出自参考文献 9）

- **可前瞻：**取回的轨迹还要能被整合成一个支持未来动作的工作状态。记忆不能只是描述过去，得为将来所用。

怎么做的？ 四步逐一落地这三条。每个时间步把 RGB 视图、本体感觉、可选的语言指令绑成一个具身事件 token，记的不只是画面、而是“视觉加本体加语言”的联合证据。这些 token 经一个慢速的情景级记忆模块往后传播，把不同历史保留成彼此区分的状态、而不是压成单一向量（这实现了可区分）。要做动作决策时，从当前具身状态导出一个学习到的控制索引，拿它去查询记忆、取回与当前这个选择相关的轨迹（这实现了可寻址）。取回的轨迹被整合成一个快速工作状态用于动作预测，并用 **Control-JEPA**（一个自监督目标，让这个状态去预测未来的控制上下文）来训练它（这实现了可前瞻，也给观测-动作延迟一个直接的学习信号）。

一句话：慢记忆保留区分、控制索引按需检索、快状态前瞻整合，串起来就是**控制索引的前瞻记忆**。

效果如何？ 论文专门造了一个诊断基准 **Camo-Dataset**：真机 UR5，刻意把决策场景做得视觉歧义，逼策略必须靠早先观测推断正确动作；而且把“记忆决策是否正确”与“动作执行是否成功”**分开计分**。Chameleon 把决策与端到端成功率从 22.5% 大幅拉高。公开长程记忆基准 (LIBERO-10、MemoryBench、MIKASA-Robo) 上达到同体量模型的 SOTA，并超过若干更大的 VLA。probe 与消融显示那三条性质确实被学到、且对性能是功能性必要的。

这三条标尺是这一节最值得带走的东西：后面每一篇记忆 VLA 都可以拿它来审视。

2.6 MemoryVLA：把记忆做成显式记忆库

解决什么问题？ 这是这条线的锚点，它把“VLA 的记忆该长什么样”做得最完整。要对付的还是操作的非马尔可夫性，最直白的例子是“按按钮”：机器人要依次按下三个按钮，可按下前和按下后画面儿

乎一模一样，只看当前帧根本无法判断“这个按钮我到底按过没有”，该信息只存在于过去。

最朴素的想法是把最近几帧拼起来喂给 VLM。这条路有两个死穴：self-attention 是二次复杂度，帧一多可用历史长度就被卡死；而且 VLA 的机器人预训练几乎都是单帧的，硬塞多帧序列会让输入分布和预训练对不上、性能反而掉。MemoryVLA 的答案是不靠拼帧，而是显式地建一个可检索、可更新的记忆库。

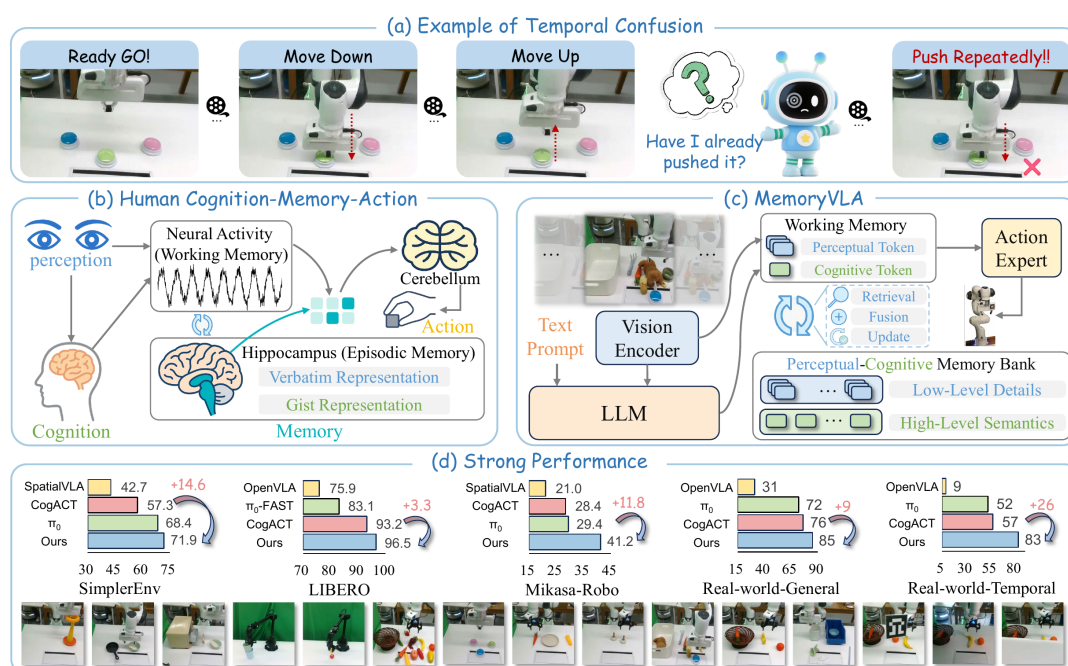


图 7: 按按钮任务的非马尔可夫性、人类双记忆系统、以及感知-认知记忆库的整体设计（图片出自参考文献 10）

怎么做的？设计直接类比认知科学里的人类双记忆系统：**工作记忆**是前额叶的瞬时神经活动，缓冲当下的感知、支持即时控制；**长期情景记忆**在海马体，把过去经历按时间索引存下来，而且存两种形式，逐字的精确细节加语义概要的抽象。人做事时工作记忆会从长期记忆里取回相关片段、融合进当前感知来指导动作，同时把新经历写回长期记忆。MemoryVLA 把这套“取回、融合、写回”照搬成一个认

知-记忆-动作框架。

认知这一段把当前观测编码成两种 token。**感知 token** 由 DINOv2 与 SigLIP 两个视觉骨干并行提特征、拼起来后用一个 SE-bottleneck 压成 256 个紧凑 token，保留低层细粒度的视觉细节（物体在哪、什么姿态）。**认知 token** 把视觉 token 投影进语言空间、和指令一起喂 LLaMA-7B，取句末 EOS 位置的输出作为一个 token，用 VLM 的常识先验浓缩出高层语义（现在处于任务的哪一步、该干什么）。两者合起来是工作记忆，但它只反映当下这一帧、没有任何时序信息，时序要靠记忆库补。

备注：一“感知”一“认知”的分工是关键设计，细节和语义**分两路存、分两路用**，后面动作专家也是分两路条件。不要把它们混成一团。

记忆这一段就是那个“海马体”：感知-认知记忆库有两条流，每条存至多 L 条带时间戳的历史条目。每来一帧做三件事。

检索：当前的感知与认知 token 作查询，每条记忆条目都加上自己的**时间戳正弦位置编码**（记住它是何时发生的），当前 token 与记忆做缩放点积注意力（两层 Transformer），取回与当前决策相关的历史表征 H^p 、 H^c 。

门控融合：取回的历史不能无脑全信，要看当下需不需要。用一个学习到的门：

$$g^x = \sigma(\text{MLP}([x, H^x])), \quad \tilde{x} = g^x \odot H^x + (1 - g^x) \odot x$$

x 是当前 token， H^x 是取回的对应历史， g^x 逐元素决定“信历史多少、信当下多少”。门开得大就多信历史，开得小就多信当下。

巩固：融合后的 $\tilde{p}$ 、 $\tilde{c}$ 写回记忆库。记忆会越积越多，所以条目数超过容量 L 时，在每条流内计算**相邻条目的余弦相似度、把最像的一对合并取平均**。这既压住了记忆膨胀，又保留了最关键的细节与语义概要，和人脑把零碎相似的经历归并成一段是同一个直觉。

动作这一段由记忆增强后的 $\{\tilde{p}, \tilde{c}\}$ 驱动一个扩散 Transformer（DDIM 10 步去噪），一次预测 16 步的 7 自由度连续动作（相对平移、相对旋转、夹爪开合）。条件方式仍是分两路：认知注意力用 $\tilde{c}$ 给高层语义引导，感知注意力用 $\tilde{p}$ 补细粒度视觉细节。一次预测 16 步而非 1 步，让模型能规划多步轨迹、降低累计误差，这和第 10 讲 ACT 的动作分块一脉相承。

效果如何？覆盖 3 个机器人、150 多个任务、500 多个变体，仿真加真机全测。SimplerEnv（Bridge 71.9%、Fractal 72.7%）、LIBERO（96.5%）、Mikasa-Robo（41.2%）全面超越 CogACT 与 π_0 ，其中 Bridge 上有 +14.6 的大幅增益。

长程才是它的主场：12 个真实任务里，6 个长程时序任务相对 SOTA 提升约 +26，而普通短程任务上差距小。这个对比反过来印证了记忆机制正是补在“非马尔可夫、需要记住过去”这个缺口上。背景、干扰物、物体、容器、光照、遮挡等分布外条件下也依然稳。

拿 2.5 节那三条标尺一对：**检索对应可寻址，双流分存对应可区分，记忆条件的动作专家对应可前瞻。**

2.7 Dual-Memory VLA：记忆还能用来提速和稳住生成

解决什么问题？这一篇 [11]（论文里叫 OptimusVLA）提供一个有意思的对照：它也叫“双记忆”，但记的东西和 MemoryVLA 完全不同，一个记**生成动作的先验**（为了更快），一个记**近期执行的动作**（为了更稳）。它让我们看到“记忆”在 VLA 里其实有不只一种用法。

它瞄准的是分层 VLA 一个被忽视的瓶颈：**动作生成本身**又慢又不够稳。主流 VLA 用扩散或流匹配，从高斯噪声出发多步去噪生成动作，两个问题随之而来。效率上，从各向同性高斯噪声映到结构化的动作分布跨度很大，所以需要很多步去噪；而且随机起点经常落在**运动学不可行**的区域，采出废样本。直接拿某个动作先验当起点又会把多样性压没、塌成“只会输出相似动作”。

怎么做的？两块记忆各治一个病。

全局先验记忆治“慢”。核心想法是把“生成的起点该是什么”从固定的噪声设计**改成一个记忆检索问题**。三件套：Prior Head 整合当前多模态信息；Memory Bank 存储并检索来自语义相似历史轨迹的**任务级先验**；Prior-Aware Sampler 从检索到的先验动作分布采样，并自适应决定噪声尺度和函数求值次数（NFE，约等于去噪步数）。直觉是与其每次都从一团随机噪声出发，不如从**“相似任务的动作长什么样”附近出发**：生成路径短了、起点本就落在可行动作空间附近、少采废样本。同时保留自适应的噪声注入，避免塌缩成“只会复制先验”，保住对新任务的泛化。

局部一致性记忆治“抖、分不清阶段”。它给 VLA 补时序感知，但不背长上下文的包袱。两个轻量结构：Consistency Layer 吃进最近几段动作块、建模局部一致性；Dynamic-Awareness 模块动态建模历史序列、推断**任务进度**（现在到哪一步了）。它学到一个一致性约束注入策略输入，强制新动作和已执行轨迹连贯、并带上进度感知。关键是它**不改 VLA 的预训练范式**、计算开销可忽略。

效果如何？3 个仿真器加真机评测，LIBERO 平均成功率 **98.6%** 超过强基线，同时带来明显的推理加速，后者正是降低 NFE 的直接好处。

2.8 从“记住过去”到“预判未来”：MemoryVLA++

解决什么问题？MemoryVLA 让 VLA 记住过去，同一批作者的续作指出：**记忆只是时序建模的一半**。两个任务把这件事讲得很清楚。按按钮需要**记忆**（我到底按过没有）；传送带抓取需要**想象**（预测物体接下来会移到哪，才能在合适的时机出手）。

最朴素的想法是输入里既塞最近 N 帧历史、又塞预测出来的未来视频帧。两头都低效。过去侧，拼接连续历史帧带来二次注意力成本和大量时序冗余，限制可用上下文长度；操作任务动作慢、关键交互被拉得很长，这个问题更严重。未来侧，预测完整未来视频既贵、又偏重**像素级保真**而非控制相关的动态，而且把预测的像素帧直接喂进 VLA，**视觉预测误差会传导到动作生成**。所以要的是“把过去压成长期记忆”加“用紧凑、决策相关的想象抓未来动态”。

怎么做的？记忆那一半沿用 MemoryVLA。新增的核心是一个**视频生成世界模型**，但它不去生成完整像素未来帧，而是在**潜空间里做部分去噪**，直接得到想象的 future token。这个设计正好对症上面两个毛病：在潜空间且只做部分去噪，所以**紧凑**，不背完整视频生成的算力；产物是 token 而非像素帧、聚焦

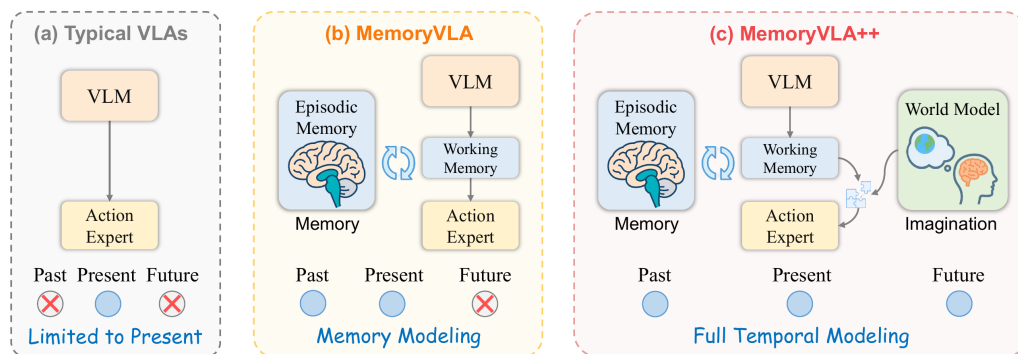


图 8: 三种范式：典型 VLA 只看当下 → MemoryVLA 加记忆 → MemoryVLA++ 再加世界模型想象未来（图片出自参考文献 12）

控制相关动态，所以**少把像素预测误差传给动作**。最后在记忆增强 token 的引导下把想象的 future token 整合进来，形成“现在感知加过去记忆加未来想象”三合一的完整时序表征，再去条件扩散动作专家。

效果如何？ 5 个仿真基准加 3 类真机任务、3 个机器人、近 200 个任务。仿真成功率约 98.4%，长程时序任务相对提升 +44.4%。真机三类收益分得很清楚：**通用任务 +9%、记忆依赖任务 +26%、想象依赖任务 +28%**。记忆和想象各自补在了对应类型的任务上，这正印证了“完整时序建模需要两半”的主张。

2.9 BagelVLA：另一条向前看的路线

解决什么问题？“向前看”不止一种实现。BagelVLA[13] 把一个通用智能体该有的三种能力——理解做什么（语言规划）、预测会发生什么（视觉预测）、执行动作（控制）——**交错统一**进同一个模型。它

的出发点是观察到现有 VLA 往往把这三种能力当成孤立模块：要么只做高层语言规划、缺视觉预测，要么只做视觉预测、缺逻辑推理。

机会来自多模态领域出现的**统一理解-生成模型**（如 Bagel）：单个 transformer 同时处理和生成文本与图像，恰好能“用文字想下一步、用像素想象结果”。但通用模型不是为具身推理和实时控制设计的，BagelVLA 要做的是把这种先验变得可用于长程操作。

怎么做的？在一个统一 transformer 里交错三步：先生成一段文本计划把指令拆解开（下一个该操作的物体是哪个），再预测未来的视觉状态，最后生成动作。这样就把语言模型的逻辑推理和视觉生成的预测力结合起来。

把视觉生成塞进控制回路最大的麻烦是**延迟**，生成一帧完整未来图像太慢、控制等不起。BagelVLA 的解法是**残差流引导**：不从零合成未来帧，而是以当前观测作强结构先验、用**单步去噪**预测“朝下一关键帧的残差变化”，从而高效提取预测性视觉特征来引导动作，省掉完整图像合成的算力。

训练分两阶段。先用通用多模态数据加大规模机器人数据的混合数据集注入具身多模态规划能力，机器人数据标注了**子任务和关键帧**；再引入动作专家微调整个模型，把语言、预测的视觉动态、控制三者耦合起来，同时保住高层推理能力。

效果如何？显式耦合语言规划与视觉预测之后，在多阶段推理与长程任务上相对基线显著领先；真机上对未见指令和多样物体摆放泛化强，而基线常常失败。

把它和 MemoryVLA++ 对照着看很有意思：两者都要看未来，MemoryVLA++ 在潜空间用世界模型部分去噪出未来 token，BagelVLA 用单步残差从当前帧推下一关键帧。**殊途同归，都在想办法让“对未来的预测”既有用又不拖慢控制。**

2.10 Long-VLA：把长程能力直接收口

无论记忆还是想象，最终都服务于**长程多步操作**。长程的难点在**技能串接**：端到端模型不擅长串接，把多个策略拼起来又失了统一。

图里四格是按“三项能占几项”排的。(a) 朴素端到端 VLA 只有一个策略吃整条长任务，**统一**这一项占住了，但它撑不住长程、也管不了技能串接。(b) 把任务拆成移动策略与交互策略两个模型，**长程**这一项拿到了，代价是统一没了——两个方框中间那道红色虚线就是接缝，交接处得另外想办法，所以串接这一项仍是空的。(c) 在两段式的基础上给每个策略的输入侧加一道**掩码闸门**，接缝靠输入级适配接住，串接这一项才补上，但仍然是两个模型。(d) 是 Long-VLA[14] 的位置：闸门保留、策略只剩一个，三项同时占齐。

关键在最下面那行说明的闸门本身。**分阶段的输入掩码**（phase-aware masking）把每个子任务切成**移动 / 交互**两段：移动段只放行远处的导航线索、遮住近处的操作线索，交互段反过来。同一个模型在两段里看到的输入不一样，等于自己扮演了两个专用策略，接缝就不必再交给第二个模型。Long-VLA 靠这一手在一个端到端统一模型里拿到技能串接能力，直接以“把长任务真正做完”为目标收口。

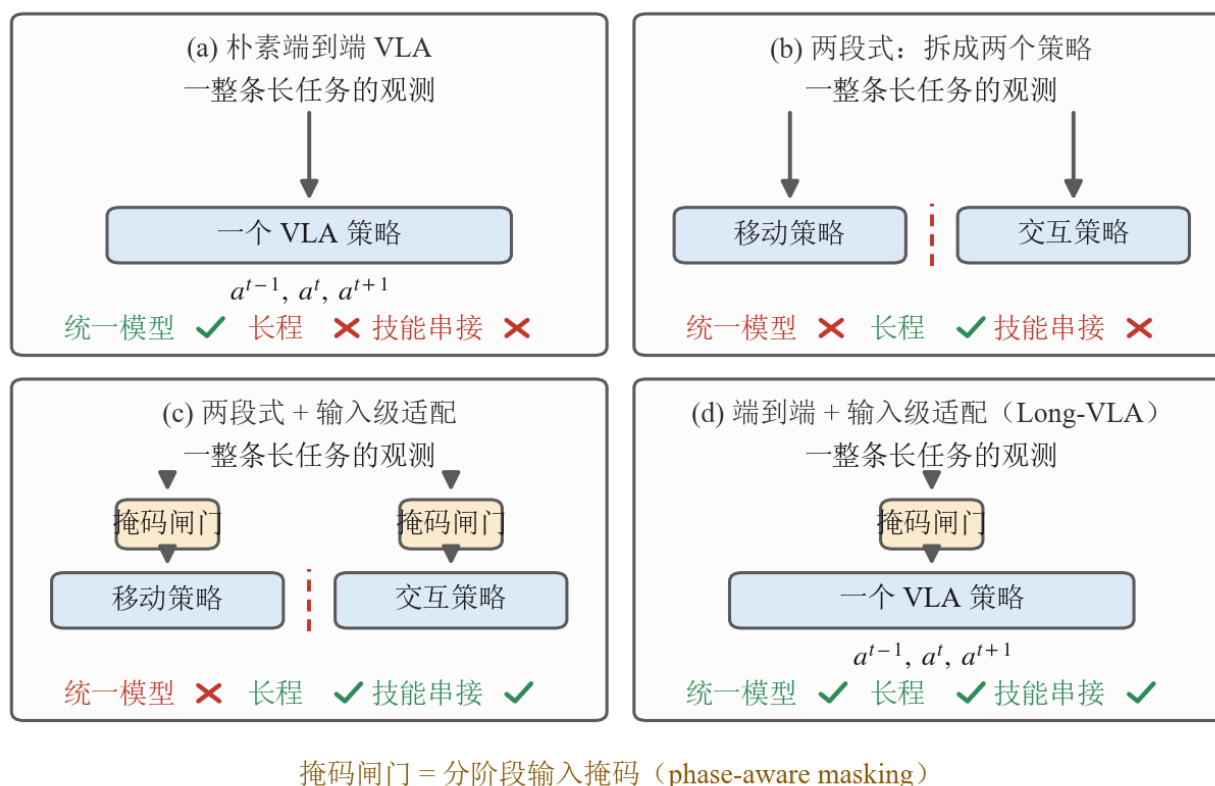


图 9: 长程操作的四种范式 (自绘, 按参考文献 14 图 1 的四类范式重画)。每格上方是这一路的数据流, 下方是“统一模型 / 长程 / 技能串接”三项的勾叉

2.11 易混点：“记忆”和“想象”各指好几样东西

八篇走完，“记忆”和“想象”在这条线里已经指了好几样不同的东西。听到这两个词时要先问一句：

- **记的是什么：**MemoryVLA 记的是**观测历史**；HAMLET 记的是每步压缩的 **token**；Dual-Memory 记的是**生成先验 + 近期动作**；Long-Context DP 根本不建记忆库，只把历史当长上下文。
- **想象怎么做：**MemoryVLA++ 是**世界模型潜空间想象**；BagelVLA 是**语言规划 + 视觉残差预测**。都想“看未来”，一个偏视觉生成、一个偏推理 + 预测。
- **记忆不等于长上下文：**把历史一股脑塞进上下文只是最朴素的一种，又贵又容易错配预训练分布——2.5、2.6 两节之所以要绕开它、去建“显式、可检索、可压缩”的记忆库，理由就在这里。

2.12 记忆的代价，以及“收益是在哪量出来的”

1.2 节给三条实时路线各记了一笔代价，这一节也得记。给 VLA 装记忆不是白装的：

- 它和第 1 节的目标直接打架。每一步都要去记忆库里检索、再和当下融合，这些计算原本不存

在。1.5 节费半天劲把单次推理从 106 ms 压到 27 ms，这一节又往每一步里加一道检索和一道融合——两条线是同一台机器上的资源竞争，不是可以各自庆祝的两项进展。

- **巩固是有损压缩。**记忆库不能无限长，所以 MemoryVLA 要合并相邻最相似的条目。可“相似”是模型自己判的：合错了，那段历史就等于没记过，而且没有任何报错告诉你它丢了什么。
- **长上下文那条路省事但更贵。**把历史一股脑塞进上下文，序列长度线性涨、注意力开销更陡，还容易偏离预训练时的分布，2.11 节说的就是这件事。

2.13 本节小结

八篇工作按“记忆做得越来越显式”排成一条线，每往前一格，能力和代价同时上涨：

记忆的形态	代表	换来什么	付出什么
不建结构，历史当长上下文	Long-Context DP	最省事，靠 PTP 正则真把历史用起来	历史是一整段窗口，无法挑选或长期保留
外挂一组 token 加轻聚合	HAMLET	不重训大模型就得到历史感知	聚合偏“汇总”，答不了“该找回哪一段”
控制索引按需检索	Chameleon	记忆可区分、可寻址、可前瞻	要额外训一个自监督的前瞻目标
显式双流记忆库	MemoryVLA	检索、融合、巩固成一套完整机制	巩固是有损压缩，合错了不报错
先验与一致性也当记忆	Dual-Memory VLA	生成更快（降 NFE）、更稳（进度感知）	先验检索本身要维护一个库
记忆之上加潜空间想象	MemoryVLA++	补齐时序建模的另一半	多一个世界模型的推理开销
规划、预测、动作交错	BagelVLA	长程与多阶段推理显著领先	统一模型的训练成本最高
阶段感知输入掩码	Long-VLA	端到端里直接拿到技能串接	掩码怎么切阶段仍需按任务定

竖着读这张表，能看出三件事。

第一，收益都集中在长程。MemoryVLA 的 6 个长程真机任务提升约 +26 而短程差距小，MemoryVLA++ 的记忆依赖 +26、想象依赖 +28 而通用只有 +9，HAMLET 的长程真机任务 +47.2%。同一个方法换一批任务，收益能差三倍。**看到“记忆让成功率涨了多少”，先问它是在什么任务上量的。**

第二，“记忆”这个词在这八篇里指的不是同一件事。观测历史、每步压缩的 token、生成先验、近期动作的一致性，都被叫做记忆。2.11 节那三个问句就是为此准备的。

第三，记忆和第 1 节的实时目标是同一台机器上的资源竞争。检索加融合的开销原本不存在，而 Dual-Memory 恰好给出一个反向的例子：它用记忆去降低 NFE。这说明“加记忆一定更慢”并非定论，取决于记忆被用在哪一环。

还有一条读论文时更该带着的警觉：**这条线上的收益，绝大多数是在专门为“需要记忆”而造的任务上量出来的**。Chameleon 干脆自己造了一个基准，把决策场景做成视觉上有歧义、必须靠更早的观测才能判断；MemoryVLA++ 在真机上报的三个数把这件事摆得很清楚——一般任务提升约 9 个百分点，而依赖记忆和依赖想象的两类任务分别是约 26 和 28 个百分点。同一个模型，换一批不吃记忆的任务，收益会缩水到三分之一。

所以从隐式长上下文，到显式记忆库，再到记忆 + 想象与长程收口，这条线确实让一个只会看当下的策略学会了用过去、预判未来。但它的正确读法要落在一个先问自己的问题上，而非笼统的“记忆让 VLA 更强”：**你的任务里到底有没有观测-动作延迟？**有，这些方法值回它的推理开销；没有，你付了成本却买不到多少东西。

3 语言直驱全身：让一句话落到整个身体

3.1 人形全身难在哪

桌面机械臂的 VLA，主要难在“看懂场景、选对动作”。换成人形机器人，难度会突然上一个台阶：它是一个会移动、会转身、会蹲下、会被上肢动作牵动重心的完整身体，不再是固定在桌边的一只手。所以人形全身 VLA 不是简单把动作维度加大，它同时遇到四个问题：

- **语言要落到全身动作**：一句“走过去挥手”要落成的是一段全身运动风格，不是末端位姿。
- **视觉要决定空间关系**：椅子在左还是右、物体远还是近，会改变身体怎么走、怎么停、怎么伸手。
- **高层语义和低层控制频率差太大**：VLA 可以慢一点想，但腿和身体必须高频稳定执行。
- **数据太贵**：人形全身遥操作、动捕、真机采集都比桌面操作贵得多。

这些难点最后都收敛成同一对矛盾：

一边是“懂任务、但不会走路的大脑”，一边是“会走路、但不懂任务的全身”，怎么把这两条线拼成一个端到端、会全身干活的系统？

把这两端拼起来是一串工作各自推进、逐渐拼出来的，所以这一讲也按两端分开讲：**本节走大脑那一端，下一节走身体这一端**。

本节四小节按**问题的递进顺序**排（这是叙述线索，不是时间线——同期工作很多是并行发布的）：3.2 先证明一句语言真能直接驱动全身，3.3 用一个潜在接口把“高层想”和“低层动”分开，3.4 补上“会移动”这一块、并让潜在动作可以从无动作视频里学，3.5 回答“这么贵的数据从哪来”。第 4 节接着从身体这一端往回修，最后在 4.4 节两端合龙。

3.2 LangWBC：先证明语言能直驱全身

这条线上最先要回答的一道题是：如果没有视觉，只有一句语言指令，能不能让人形直接做出全身动作？LangWBC[15] 没有走传统的“语言 → 参考轨迹 → 跟踪控制器”路线，而是训练一个最终部署用

的 **CVAE Student**——CVAE 是条件变分自编码器 (Conditional Variational Autoencoder)，一种能按给定条件采样出多样输出的生成模型。这个 Student 输入语言和本体感知，直接输出全身动作。

它在反对什么。要理解这个设计，得先知道它绕开了哪条路。在它之前，语言驱动人形控制的主流是**分层**：先用一个运动生成模块把语言变成一段**参考运动轨迹**（每一帧机器人该长什么样），再用一个全身跟踪策略去跟踪它。这条路有两处硬伤。生成的轨迹**常常物理上不可行**，因为运动生成模块只懂“动作好不好看”、不懂“机器人能不能站住”，于是会生成下肢悬空、重心超出稳定边界的姿态；跟踪策略夹在中间很为难，跟得准就要摔倒、站得稳就跟不准。而且这类方法处理的是一段段**定长片段**，难以应对中途扰动、也难以在两个动作之间平滑切换。

LangWBC 的主张很直接：**取消中间那段显式参考轨迹**，让“动作生成”和“物理可行”这对矛盾在同一个网络里被一起优化掉，而不是交给两个互相打架的模块。

两阶段的分工。一句话概括是“先学会动，再学会听”。两件事的难点完全不同：学会动需要强化学习和物理引擎反复试错，需要摩擦、质量、外力这些只有仿真里拿得到的**特权信息**；学会听则要把语言对齐到动作，且最终必须只靠机器人自带传感器运行。拆开各用最合适的工具，是这个框架的核心思路。

第一阶段的 Teacher 只做一件事：精确跟踪人类动作，不涉及任何语言。数据来自 HumanML3D 这个带文字描述的人类动捕数据集，但人的骨架和 G1 的关节结构对不上，所以先做**运动重定向**：用 Levenberg-Marquardt 解一个非线性最小二乘，让机器人的关键点位置和姿态尽量贴近动捕关键点，同时加上平滑与关节限位约束，得到一批**运动学上可行**的参考动作。然后用 PPO 训一个三层 MLP (512 / 256 / 128) 去跟踪：

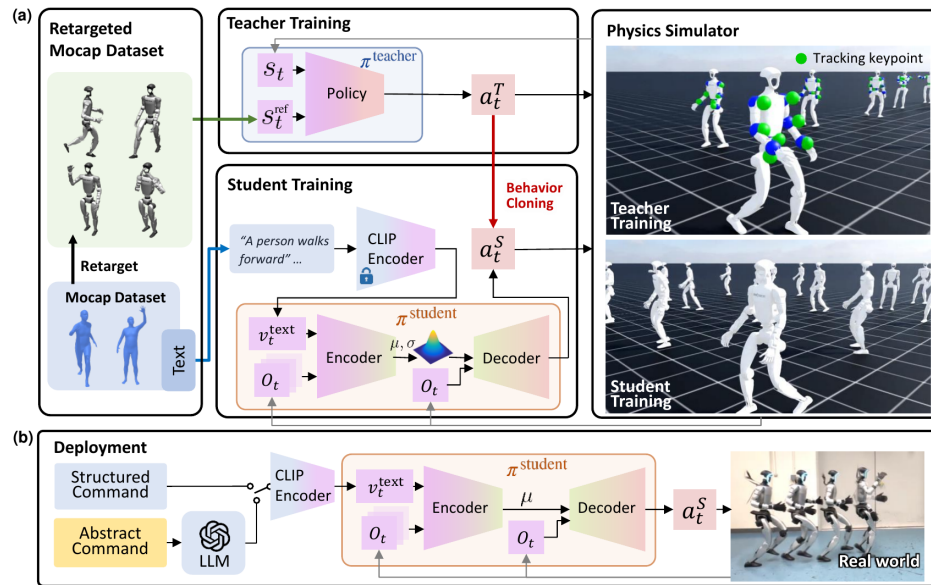
$$a_t^T = \pi^{\text{teacher}}(s_t, s_t^{\text{ref}})$$

s_t 是 175 维机器人状态（含本体感知**和特权信息**）， s_t^{ref} 是 141 维参考动作（含未来 5 帧的关键点位置与参考关节角），输出 27 维目标关节角交给底层 PD 控制器，跑在 50 Hz。

两个训练技巧值得记住。**运动课程**：动捕里有大量敏捷动作，一上来全喂会让梯度方差爆炸，所以先学静态与准静态的“简单”档，跟得好了再逐步加入急转、快跑这些“困难”档。**对称损失**：人体动作天然左右对称，于是对每个状态-动作对生成左右镜像版本，约束策略对镜像状态输出镜像动作，既让动作更自然平衡、又省样本。

第二阶段的 Student 才是部署时真正用的策略，它有两个要求：只靠本体感知运行（这样才能上真机），以及以文本指令为条件。LangWBC 用一个 CVAE 一并解决。对应关系是：条件 c 取 CLIP 文本嵌入加最近 2 秒的本体观测历史，目标 x 取 Teacher 在当前状态下给出的动作，隐变量 z_t 是 128 维的动作模式编码，解码器输出当前帧动作。

输入的两部分都具体：文本指令过 CLIP 文本编码器成一个 512 维语义向量；本体观测只用机器人自带传感器拿得到的 90 维（关节位置与速度、base 线速度与角速度、投影重力），取 2 秒、10 Hz、共 20 步的历史。**没有特权信息**，这正是它能零样本上真机的原因：真机上拿不到摩擦和外力，但本体传感器永远有。



S_t Proprioception State $\hat{S}_t$ reference State a_t Proprioception Action $\hat{a}_t$ Reference Action

图 10: LangWBC 两阶段框架：先训练只会跟踪的 Teacher，再蒸馏成以语言为条件、只靠本体感知运行的 CVAE Student（图片出自参考文献 15）

三步写下来是：

$$\mu_t, \sigma_t = \pi_{\text{enc.}}^{\text{student}}(o_t, \dots, o_{t-20}, v_t^{\text{text}}), \quad z_t = \mu_t + \sigma_t \odot \epsilon_t, \quad a_t^S = \pi_{\text{dec.}}^{\text{student}}(z_t, o_t)$$

编码器是 MLP (2048 / 1024 / 512)，解码器镜像对称。有一个关键工程细节：**训练时**用重参数化采样 z_t （带噪声，保证隐空间被填满、连续），**推理时直接取均值 μ_t 、丢掉采样**，这样同一句指令每次给出确定性动作，部署才稳。

蒸馏用 DAgger，损失就是变分下界的标准形式：

$$\mathcal{L}_{\text{student}} = \|a_t^T - a_t^S\|_2^2 + \lambda_{\text{KL}} D_{\text{KL}}(q_\phi(z_t | o_{t-20:t}, v_t^{\text{text}}) \| p(z_t))$$

第一项是行为克隆的重建项，让学生的动作贴近老师的。**第二项那个 KL 正则是全篇的灵魂**：重建损失会逼 z 携带动作信息，KL 会把 z 往标准正态附近拉。KL 像一根弹簧，太松隐空间散成一堆孤岛、太紧所有动作压成同一个点；合适的权重才得到“既有语义区分、又能平滑过渡”的隐空间。后面所有涌现能力都来自这个结构。

还有一个稳住蒸馏的技巧。蒸馏初期学生还很笨，累计误差会把机器人推到 Teacher 从没见过的状态，而 Teacher 一走出自己的训练分布，它给出的“标准答案”就不可靠、蒸馏会崩。LangWBC 的处理是让学生跟踪**相对位移**而非绝对位置： $\min \|\Delta p_t - \Delta p_{\text{ref},t}\|^2$ 。这样即使机器人整体位置偏了，只要每一步的位移趋势对得上，就还在 Teacher 的有效分布内。

效果如何？论文把结构化隐空间换来的能力逐个验证了。

对 9 个动作（走路、举左手、举右手、拍手四类）的隐编码做 t-SNE，能看到三条规律：同类动作聚成簇；举左手和举右手关于中心**镜像对称**（这正是第一阶段对称损失种下的因）；所有动作都从原点附近出发又回到原点附近，那块公共区域可以理解为“站立”这个共同的起止姿态。

正因为隐空间是这样一张规整的连续流形，三种能力自然涌现。**对未见措辞的泛化**：把 walk 换成 move 这类差异不影响执行。**动作自动成环**：参考里只有“转身一次”，策略却能连续转两次，它自己推断出了衔接时机，得益于隐空间里“站立”公共区域让动作首尾本就靠得近。**插值合成新动作**：在隐空间里对“向前走”和“左右横移”做线性插值，解码出了训练集里根本不存在的**斜向走**。

两个对照实验说明这份能力的来源。把 CVAE 换成普通 MLP（即 CLIP+MLP），未见指令上的泛化下降；直接在 CLIP 文本嵌入空间里插值，机器人会抖动、走不稳。**那份平滑可插值的结构来自 CVAE，而不是 CLIP**。消融进一步拆掉对称损失、相对位移跟踪、CVAE 架构三个设计，完整方案在动作质量（96.2%）、稳定性（99.1%）、模仿损失（0.085）三项上全面占优。值得强调的是 CVAE **不只提升泛化**，连跟踪精度和鲁棒性也比 MLP 版本更好。

鲁棒性上，机器人在挥手的同时被踢、被推，仍能保持平衡且不中断当前动作。最后一块拼图是接一个 LLM 做高层规划：LLM 把抽象场景（“前方 3 米有一位朋友”）拆成“（指令）：（时长秒数）”格式的原语序列逐条执行。这里有个关键点，LLM 生成的指令往往和训练数据相近但不完全一样，能不能容

忍这种细微差异直接决定 LLM 集成成不成，而这正是那份对相近语义的泛化在发挥作用。**结构化隐空间是 LLM 集成能跑通的前提。**

它没解决的那一半。论文自己列了三条：语言条件下的动作数量有限，受算力所限只覆盖几十种；VAE 表达力有限，存在一定 sim-to-real 差距；而最关键的一条是**没有视觉模块**，所以例子集中在移动类的全身控制，做不了需要看见物体的移动操作。LangWBC 立住了端到端、纯语言、CVAE 结构化隐空间这三个锚点，但它始终只听得见、看不见。下一节的 LeVERB 出自同一个团队，沿用同样的 CVAE 思路，在两个方向上往前走：加入视觉，以及改走分层加潜在接口的路线。两者构成一组干净的对照：**纯语言对视觉-语言、端到端对分层潜在接口。**

3.3 LeVERB：高层看图、低层稳执行

LeVERB[16] 把视觉补了进来：机器人不只听语言，还要看第一视角图像。“坐到椅子上”这句话，椅子在哪、朝向如何，必须由视觉决定。它的核心是快 / 慢双系统。

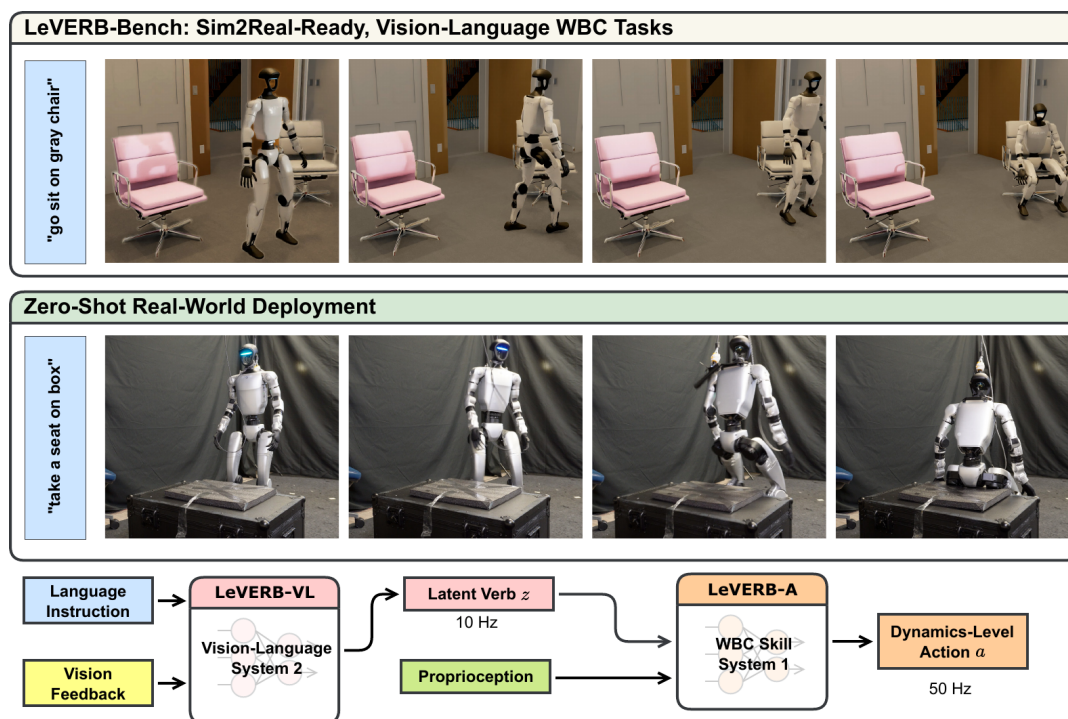


图 11: LeVERB 双系统：System 2 在 10 Hz 看图像和语言、输出潜在 verb；System 1 在 50 Hz 把潜在 verb 解码成全身动作（图片出自参考文献 16）

解决什么问题？纯语言有一个天花板：**很多全身行为必须“看着环境”才能完成。**“走到你前面那张桌子前”“坐到那把绿色的椅子上”“绕过地上的箱子走过去”，这些指令里的桌子、椅子、箱子都是**视觉指代**，机器人必须先看到它们在画面里的位置，才知道往哪个方向走、走多远、转多大角度、最后怎么把身体落进椅子里。纯语言的 LangWBC 对这些任务无能为力，它根本不知道椅子在左边还是右边。

那为什么不干脆做一个端到端的 VLA，输入图像和文字直接输出全身动作？因为人形全身控制有两个

绕不开的硬约束。**高频控制**：人形是高维强非线性的动力系统，低层动作必须以 50 Hz 这样的动力学级频率持续输出才能维持平衡，一个庞大的视觉-语言大模型根本跑不到这个频率。**训练回路里的渲染开销不可承受**：低层控制器要靠强化学习在仿真里大规模并行训练，如果策略输入含图像，RL 的每一步都要对成千上万个并行环境做照片级渲染，计算上不可行。

怎么做的？灵感来自一个朴素观察：**人是又快又慢地思考的**。你走向一把椅子时，皮层负责“我要去那把椅子”，而维持平衡、迈步、调整重心这些事由脊髓和皮层下运动回路飞快地自动完成，你根本不会去“想”每一步该怎么迈。照搬这个分层：慢系统低频看图像读指令做语义推理，快系统高频只看本体感觉做反应式全身控制。

关键问题是**这两个系统之间该传什么**。回到人脑的类比：皮层把“去椅子那儿”这个意图传给运动回路时，既不是把这几个字一个个读出来（词嵌入），也不是给出一串精确的速度指令或关节角（动力学状态空间里的指令），而是一种介于两者之间、两个系统**共享且学习得到的抽象运动意图**。LeVERB 把它叫做**潜在 verb**，记作 z 。可以把所有可能的 z 理解成一本**动作词表**，每个 z 是词表里的一个词，编码了一段复杂全身运动的目标，但又不是任何具体的关节轨迹。

备注：为什么一定要潜在接口？此前的 VLA 驱动全身控制（NaVILA、HumanPlus 那条线）用的是**显式、低维**的动作词表，比如底盘速度、末端位姿。这种接口好实现，但**表达力受限**：用“底盘速度”这种词汇，你很难表达“坐到椅子上”这类需要全身协调的行为。学习得到的潜在词表既能覆盖丰富的全身动作，又能承载视觉-语言里的语义。

推理时的完整策略写成：

$$\pi_{\theta}(a_t | o_t) = \int \tau_{\theta_A}(a_t | z_t, o_t^{\text{prop}}, a_{t-1}) \cdot \rho_{\theta_{VL}}(z_t | I_t, c) dz_t$$

高层 ρ_{VL} （LeVERB-VL）在 10 Hz 根据第一人称与第三人称两路图像加一句指令给出 z 的分布，低层 τ_A （LeVERB-A）在 50 Hz 根据这个 z 连同本体感觉给出动作。 z 是**唯一的桥**。正因为视觉-语言信息和动力学级动作信息被这个潜在接口彻底解耦，两个系统才可以**分开训练**，而这正是绕开渲染开销那条硬约束的关键：低层策略的训练回路里压根不出现图像。

高层怎么学出这本词表。LeVERB-VL 的整体结构是一个**残差 CVAE**，与 LangWBC 一脉相承，但输入端加了视觉、并加了三个专门部件。把它当标准 CVAE 看，角色对应关系是：语义条件取图像加指令，解码条件取当前状态 s_t ，重建目标取未来运动 $s_{t+1:t+M}$ ，隐变量就是潜在 verb。

主干叫 VLA prior，三块组成：视觉编码器用 SigLIP 的视觉分支（ViT-B/16，**全程冻结**），第一人称和第三人称图像各自独立过它、经注意力池化得到两个图像 token；文本编码器同样来自 SigLIP；再把语言 token 与两个图像 token 拼成序列送进 Transformer 主干，最后一个 MLP 头输出潜在分布的 $(\mu_{\rho}, \sigma_{\rho})$ 。用 SigLIP 是因为它的视觉与文本编码器是**对比预训练**得到的，两者嵌入天然语义对齐。这里**只用当前帧、不引入时间历史**，是一个刻意的反过拟合设计：合成数据规模有限，喂历史容易让模型记住轨迹而非理解语义。

三个部件各治一个问题。

运动学编码器 E_ψ 补细粒度运动信息。主干只看当前帧、看不到未来该怎么动，于是加一个 MLP 编码器看**特权信息**（未来若干步的真实状态），输出另一组 (μ_E, σ_E) 。两路用**残差**方式合起来：

$$\mu = \mu_\rho + \mu_E, \quad \sigma = \sigma_E$$

这个设计的妙处是分工：主干负责“想做什么”的语义， E_ψ 只补语义里推不出的“具体怎么动”。再加一个 KL 约束把后验往先验拉，逼着 E_ψ **只承载视觉-语言无法推断的那部分**。KL 项像一根弹簧，把编码器这一路的输出往“主干已经能推断出来的部分”收：凡是看图听话就能知道的，不许编码器再单独编码一遍。这样训完之后丢掉 E_ψ 、只用主干先验，潜码依然有意义。

运动学解码器 D_ψ 给潜空间一个学习信号。从后验采样得到 z_t ，连同当前状态一起喂给 MLP 解码器去重建未来状态。这个**轨迹重建损失**就是整个高层学习的主信号：能重建出未来轨迹，说明 z 真的编码进了运动意图。

判别器加梯度反转层对齐“盲”与“非盲”数据。这是最巧妙也最关键的一个部件，解决的是数据问题。训练数据有两类：带视觉的视觉-语言轨迹，以及**纯语言轨迹**（没有图像，便宜、量大）。混在一起训的办法是给纯语言轨迹喂一张**白噪声图像**、假装它“盲”着。可这样一来两类输入产生的潜码会落在潜空间的不同区域、形成分布偏移，纯语言学到的运动技能就迁移不到视觉任务上。于是引入一个判别器，拿潜码去猜这条数据到底有没有真实图像，用二分类交叉熵训练。

梯度反转层前向什么都不改，反向时**只把传回主干的判别器梯度乘以负号**，判别器自己的参数仍然正常更新。所以同一项损失对两边的更新方向不同：判别器越来越会分辨盲与非盲，而主干被训练成让判别器更难分辨。

这里最容易误解：**GRL 不是让主干变差**。主干对轨迹重建项和 KL 项仍然正常梯度下降， z 必须保留动作意图，否则解码器重建不了未来轨迹、重建损失立刻变大。GRL 擦掉的只是 z 里的**数据来源痕迹**。训练目标是“让 z 没有数据来源信息”，不是“让 z 没信息”，最终得到一种**对动作有用、但对是否有真实图像不敏感**的潜在 verb。

总损失把三件事合起来：

$$\mathcal{L} = \underbrace{\|D_\psi(s_t, z) - s_{t+1:t+H}^R\|_2^2}_{\text{轨迹重建: 保留动作意图}} + \underbrace{\beta_1 D_{\text{KL}}(q(z_t) \| p(z_t))}_{\text{让编码器只补推不出的部分}} + \underbrace{\beta_2 \mathcal{L}_{\text{disc}}(z_t)}_{\text{对齐盲与非盲潜空间}}$$

后验 $q(z_t) = \mathcal{N}(\mu_\rho + \mu_E, \sigma_E^2)$ 看到了未来轨迹，先验 $p(z_t) = \mathcal{N}(\mu_\rho, \sigma_\rho^2)$ 只看视觉-语言。KL 把前者往后者拉，正是“让主干学会光凭视觉-语言就给出好潜码”的训练机制。部署时未来轨迹、运动学编码器、解码器、判别器全都丢掉，留下的只有“图像加文字 $\rightarrow$ 潜在 verb”这条通路。

低层怎么学会执行。高层训完后**冻结它的潜空间**，再训低层。低层是个 50 Hz、只看本体感觉不看图像的反应式全身控制器：先用 PPO 训一批只跟踪运动的 teacher，再冻结潜空间、用 DAgger 把多个 teacher 蒸馏成一个以 z 为条件的 student。teacher 看的是参考运动，**不看图像、不看语言、也不知道**

z 是什么，它只回答“在当前状态下要跟好这段参考运动，专家动作该是什么”。student 反复学之后就掌握了“看到某个 *z* 该调动怎样的全身动作”。

顺手造了一个基准。训练高层需要成对的“机器人视频加语言”数据，而这种数据此前根本不存在，所以 LeVERB 一并造了 **LeVERB-Bench**：第一个 sim-to-real-ready、视觉-语言、闭环的人形全身控制基准，覆盖 10 类、150 多个任务。思路是把人类运动捕捉动作重定向到人形身上，在仿真里**纯运动学重放**（不需要真实动力学控制、也不需要专家策略，省算力，论点是纯运动学姿态已经含有足够的任务级语义），用 IsaacSim 的光线追踪渲染缓解视觉的 sim-to-real 差距，再用 VLM 自动标注第一人称文字指令。规模是 **154 条视觉-语言轨迹、每条随机化 100 次、渲染出 17.1 小时照片级 rollout**，另加 **2.7 小时纯语言数据（500 条轨迹）**。纯语言那批不需要渲染、便宜，靠在文本里注入空间线索（“左边那把红椅子”）弥补没有图像带来的歧义。这两类数据正是判别器要去对齐的非盲与盲。

效果如何？整体成功率 **58.5%**，简单视觉导航（目标就在正前方）80%，10 类任务里有 9 类夺冠。对照一个**朴素分层 VLA 基线**（两个系统都不采样、对高层输出做确定性条件）高出约 **7.8 倍**（58.5% 对 7.5%）。这一条本身就说明：把视觉-语言分层接到全身控制上，**朴素地接是接不通的**。

四个消融各证明一个部件：

消融	去掉了什么	成功率	说明的事
完整 LeVERB	—	58.5%	—
NE	运动学编码器	53.0%	主干被迫把细粒度运动也编进潜空间，语义被稀释、泛化变差
ND	判别器	33.0%	盲与非盲潜空间不对齐，纯语言技能迁不到视觉任务， 视觉导航大幅下滑
NVL	高层 VL（直接条件低层）	25.5%	退化成 LangWBC 式做法，纯语言任务尚可， 视觉任务几乎全崩
NS	两系统都不采样	7.5%	潜空间无结构、插值差、低层频繁跑出分布

NVL 那一行最值得玩味。它把视觉与语言嵌入直接喂给低层、绕过高层规划，这正是退化成了 LangWBC 式的“直接条件低层”。结果是纯语言任务还行（移动类甚至 100%），可所有视觉任务几乎全军覆没。这条消融给出了这两节最重要的论证：**原子语言指令确实不需要高层推理就能学会（与 LangWBC 一致），但一旦引入视觉反馈，低层策略本身就不够了，必须有一个视觉-语言高层来做规划。**

真机上 LeVERB 在 Unitree G1 完成了动力学级零样本 sim-to-real：把高层闭环输出的潜在 verb 记录下来，在真机上开环回放给低层执行，成功完成视觉导航与坐下。两点泛化值得记：**词表泛化**，给出训练中未见过的动词-物体组合（“rest on the box”）仍能输出正确的潜在 verb；**空间推理**，把目标椅子放在相对机器人不同的位姿，机器人能依据视觉反馈走对方向、转对角度，最后稳稳落进椅子里。

高层负责“看懂要做什么”，低层负责“把身体稳稳做出来”，中间只传一个潜在 verb。这个潜在 verb 是一段动作意图的压缩表示，既非自然语言单词、也非关节角。这一讲后面还会反复见到的“潜在接口”，最干净的范例就是它。

图里那两个频率要读准：10 Hz 是高层重新看一眼场景、吐一个新 verb 的节奏，50 Hz 是低层把 verb

展开成关节命令的节奏，两者都不是电机力矩环的频率。整本书遇到快 / 慢双系统都按这个口径读（同第 11 讲 3.3.4 节）：高层刷新率、动作块生成节奏、底层控制器执行频率是三个不同的量，别拿一个 Hz 概括整套系统。

LeVERB 让视觉语言任务终于能接到人形全身动作上。这个“快慢双系统 + 一个潜在接口”的骨架很好用，同期也有别的工作把它推向更大规模。

3.4 WholeBodyVLA：移动服务操作

低层的身体和它的接口都立住了，回到大脑这端——它的能力也在同步往前补。第一块是“移动”。桌面操作默认手已经在物体旁边，可人形先得走过去——而且不是走到附近就行。WholeBodyVLA[17] 把这一步提成核心问题，叫**面向操作的移动**（manipulation-aware locomotion）：人形要边走边为操作创造条件，走到某处并不算结束。论文为此专门训了一个面向 loco-manipulation 的强化学习策略，只求把前进、转身、下蹲这三件事做准做稳——因为它们正是“把身体摆到能干活的位姿”最常用的三个动作。移动不再是“先导航到附近再开始操作”，而是主动把整个身体带到适合操作的位置和姿态。

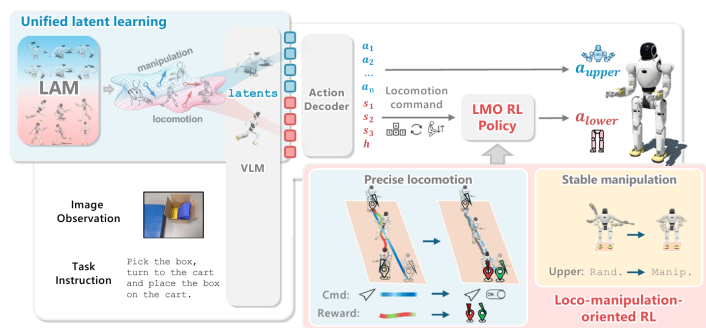


图 12: WholeBodyVLA 总览：左上角 LAM（潜在动作模型）把无动作视频压成操作 / 移动两片潜在空间；VLM 吃进图像与任务指令，输出的潜在码经 Action Decoder 分成两路——上肢动作直接下发，下肢那路是一串移动命令，交给专门训练的 LMO 强化学习策略执行（图片出自参考文献 17）

什么叫“面向操作的移动”。让一个人把桌上的纸袋装进旁边的纸箱，他不会先随便走到附近、再愣住

想下一步。他在走的过程中身体就已经在抓取做准备：朝着纸袋的方向走、走到一个手能舒服够到的距离、把身子站稳，甚至边走边微微下蹲。**移动本身就在为操作创造前提条件**。机器人却很容易把两件事割裂：先导航到某个点、停下、再交给抓取模块，而停下来那个位姿很可能对接接下来的操作极不友好，离物体太远、朝向偏了、重心不稳，结果工作空间被严重限制。所以论文的核心主张是移动要主动为操作铺好三件事：**靠近、调整朝向、稳定身姿**。

两个拦路的根因。论文很诚实地分析了为什么以前做不到。**数据稀缺**：桌面操作有海量数据（OXE、AgiBot World），轮式与四足导航也有不少数据集，可把“人形移动加操作”整合在一起的数据几乎没有；要采这种数据，动捕或遥操作都贵得离谱。**底层执行不可靠**：现有的 RL 运动控制器几乎都用**连续速度跟踪**做目标，这对大范围巡航够用，但对 loco-manipulation 需要的精确启停、刹车精度、朝向保真就力不从心，容易绊脚、路径偏移、该直走时却边转边走。关键在于**这些失败来自控制器本身、而不是高层决策错了**：高层说“往前走”，底层却走歪了。

两个组件分别正面回应：统一潜在学习对付数据稀缺，一个面向 loco-manipulation 的 RL 策略（LMO）对付执行不可靠。

统一潜在学习：让无动作视频也能教 VLA。我们想用海量“人戴着相机走来走去、靠近物体”的第一视角视频预训练，可这些视频里只有画面、没有动作标签。潜在动作学习（承自 Genie、LAPA、UniVLA）正是为绕开这个障碍而生：既然没有显式动作标签，就把**相邻两帧之间的视觉变化压成一个紧凑的离散潜在动作 token**，用它当伪标签监督策略。负责提取的模型叫 LAM，它不是控制器，更像一个“视觉变化翻译器”：看两帧图像，判断中间最可能发生了哪一类离散动作。

具体是 VQ-VAE，编码器搭在 DINOv2 特征上，四步：编码相邻两帧的变化得到连续向量 $z_t = \mathcal{E}_i(o_t, o_{t+k})$ ；到学出来的码本里找最近的离散码 $c_t^i = \arg \min_{c \in \mathcal{C}^i} \|z_t - c\|_2$ ；解码器只拿前一帧和这个离散码去重建后一帧 $\hat{o}_{t+k} = \mathcal{D}_i(o_t, c_t^i)$ ；用重建误差训练。能还原出后一帧，说明这个码确实抓住了“中间发生了什么变化”。整个训练不需要任何动作标签。

一个容易踩的坑：移动 LAM 和操作 LAM 必须分开训。因为两类视频“画面为什么变”的来源根本冲突。操作视频里相机几乎不动，画面变化几乎全来自**手臂运动**，模型会被引导去盯手臂区域；移动视频里相机持续在动，画面变化主要来自**环境相对于移动相机的位移**，模型必须去看整个场景。这两种“该往哪看”的注意力目标互相打架。硬塞进一个共享 LAM 会出一个很糟的混淆：移动视频里手臂常常也在画面里，此时“手臂和环境的相对位置变了”其实是**相机在动**造成的，共享 LAM 很可能把它误编码成“手臂在动”，潜在码一旦语义模糊，下游就学坏了。

两个 LAM 训好后就成了离线标注器，对同一批语料各给一个码。VLA 的任务是给定第一视角观测和语言指令**同时预测这两类码** $\pi_\theta(c_t^{\text{mani}}, c_t^{\text{loco}} | o_t, \ell)$ 。这一步是“统一”二字的落点：**因为是在同一个模型、同一份语料上联合预测两套码，模型被迫在一个连贯的潜在动作空间里学会移动和操作怎么相互配合**，什么时候该一边靠近一边伸手、什么时候该先站稳再抓。这是一个空间同时表达两者的交互，不是两个分开的接口拼一起。

再接一个**轻量执行解码器**，在真机遥操作轨迹上微调，把潜在码连同机器人状态翻译成两样东西：上肢关节角直接给双臂执行；下肢那路是一条“该往哪动”的高层指令，交给 LMO 落地。高层加解码器约 10 Hz，下肢指令交给 LMO 以 50 Hz 执行。

LMO：把速度跟踪换成离散启停指令。速度跟踪的问题在于启停语义是隐式的（“停”只是目标速度恰好为 0，控制器没有明确的“刹车”概念）、不同速度下步态会碎裂、而且对刹车精度与朝向保真这些 episode 级可控性几乎没有监督。可 loco-manipulation 其实不需要“任意连续速度的巡航能力”，它需要干净利落的启停、转向和下蹲，**速度跟踪给的能力是过剩且错位的。**

LMO 的核心改动是换成一个离散指令接口： $u_t = [s_x, s_y, s_\psi, h^*] \in \{-1, 0, 1\}^3 \times \mathbb{R}$ ，三个三元标志分别管前后、侧向、转向的启停，显式表达“前进 / 后退 / 停”这种语义，再加一个连续的站立高度控制下蹲。比起连续速度，它大幅降低了轨迹方差。为避免三元标志突变造成冲击（从 0 直接跳到 1 会让机器人猛地一冲），先对标志做指数平滑、再过一个饱和函数，把离散意图转成平滑的速度参考，开关切换时加速度被隐式限住。

训练用两阶段课程。第一阶段只求站得住走得稳：每个轴随机采目标速度学一个不摔的基础步态，上肢按固定间隔重采位姿目标当扰动，并用课程因子逐步放宽关节限位、给腿部施加越来越强的扰动。第二阶段针对 loco-manipulation 做三件事：把每个轴的巡航速**固定成常数**以抑制没有转向意图时的航向漂移，再用一个终端航向偏差惩罚逼策略做到精确起步、稳定巡航、一致刹车；**注入真实手臂运动扰动**，从 AgiBot World 重放真实手臂片段施加给上身，逼腿去补偿真实任务里**结构化的惯性耦合**（抓、举、推时手臂甩动对身体的反作用）而不是无结构的随机扰动，这是它比一般域随机化更扎实的地方；以及**静止惩罚**，三个意图标志都为 0 时给腿部动作一个惩罚，避免站着的时候腿乱晃。

效果如何？评测在 AgiBot X2 双足人形真机上，三个任务覆盖全部核心 primitive：装袋（抓纸袋、侧步、下蹲放入纸箱）、装箱上车（下蹲抓箱、转身放上小车）、推车（抓住车把把 50 kg 以上的小车推走）。三任务平均成功率 **78.0%**，超过导航辅助的 Modular Design (64.0%)、OpenVLA-OFT w/ LMO (56.7%)、GR00T w/ LMO (42.0%)。

四条消融把功劳分得很清楚。**潜在监督是性能的核心来源**：去掉 LAM 预训练，成功率从 78.0% 掉到 **39.3%**，整整 38.7 个百分点。**移动 LAM 不可省**：只用原地操作做潜在预训练比完整预训练低 14.7%，而且差距最大的**恰恰是那些“要先大幅移动再操作”的任务**，从反面印证了面向操作的移动的价值；共享单一 LAM 的变体 (66.0%) 略逊于分离式，说明“必须分开训”这个设计有益、但不是主因。**潜在预训练大幅省遥操作数据**：用超过 50% 的人类视频做预训练，只要 **25 条**遥操作轨迹就能匹敌那些用不到 25% 视频预训练、却需要 200 条轨迹才达到的效果。**LMO 决定成败**：换成速度跟踪变体成功率低 24%，而这个差距里 **91.7% 来自每个任务中含移动最多的第二个子目标**。这条结论的意义是：**底层控制器看似是个小问题，却是长程多步 loco-manipulation 可靠性的命门。**

它和 4.3 节 GR00T N1 内部“给无动作视频打伪标签”**共享同一个动机**（都想用无动作视频扩数据、绕开昂贵的真机动作标注），但技术实现并不相同：

	GR00T N1	WholeBodyVLA
伪标签来源	在异构数据金字塔里给无动作视频打潜在动作伪标签	由相邻帧画面变化学出离散潜在动作码本
训练目标与下游用途	潜在动作作为预训练监督，服务统一的双系统动作生成	显式预测操作 / 移动两路潜在码，再各自解码成上肢动作与下肢移动指令

所以别把它们读成“同一个思路”就带过去，latent 的语义和它最终被怎么用，是两者真正的分水岭。还有一条更容易混的：N1 那套潜在动作只活在**训练期**，用来给无动作视频造监督信号，推理时并不存在；而 3.3 节 LeVERB 的 latent verb 是**推理期**高层与低层之间真正传输的东西。同样叫 latent，一个是数据工具，一个是运行时接口。

它没解决的那一半。论文自己坦言在**超长程和高灵巧度**任务上仍有困难，缺少地图与记忆做更长的规划、缺少主动感知应对杂乱动态环境。而它那把最根本的钥匙——用潜在监督把昂贵的遥操作数据需求压下去——是从**算法接口**这一侧解决数据问题的。下一节的 Humanoid-VLA 从**另一侧**切入：与其想办法少用数据，不如换一种方式把人形全身数据搭起来。一个省数据、一个造数据，合在一起才看得清“人形全身 VLA 缺数据”这个矛盾的全貌。

3.5 Humanoid-VLA：数据规模化

最后一块是“数据”。前面三篇都默认有配好的指令-动作数据可训，可人形全身数据偏偏是最贵的一种。Humanoid-VLA[18] 换到数据视角，回答“训练信号从哪来”：再漂亮的架构，没有数据也训不出来。

它要的数据长什么样、为什么拿不到。人形 VLA 最理想的训练数据，是“机器人用自己的眼睛看到场景，同时记录下自己全身怎么动”的成对数据。可现实里两条路都断着：**动捕数据**（AMASS 这类）只有人体骨架的运动，**没有同步的第一人称视觉**，它告诉你“人怎么动”、不告诉你“人当时看到了什么、为什么这么动”；**遥操作**理论上能同时采到视觉和动作，但一套人形遥操作系统采一条数据极贵、规模根本上去。

论文开篇那张对比图刻画的是能力的跃迁。以往的全身控制是**被动反应式**的：给它一段人体动作或一句指令，它高保真地复现出来，可它不会自己看环境、不会推断潜在的交互目标。Humanoid-VLA 要的是机器人凭第一人称视觉自主感知、主动决定“该走向哪个物体、做什么动作”。要训出这种能力，就必须先解决数据。

怎么做的？主线一句话：**把“学会动作”和“接上视觉”拆成两步，因为前者根本不需要视觉数据。**人形“怎么动”靠的是海量人体运动数据，这类数据网上有的是、唯独缺第一人称视觉；而“看到场景该做什么”才真正需要 egocentric 视觉、这类数据极少。于是顺着数据的多寡分阶段。

第一阶段，语言-运动预对齐（完全不看视觉）。借鉴多模态大模型的经验：那些模型强，是因为底座有一个强大的语言模型，再把别的模态对齐进语言空间。这里把同样的思路搬到运动上，**把动作也变成语言模型能读的 token。**

具体做法是**组合式量化**：不把整具身体当个整体量化，而是把每一帧姿态拆成**左腿、右腿、躯干、左臂、右臂**五个部位，每个部位独立训一个编码器和一本码本，各自做 VQ-VAE 式离散化，一帧姿态压成 5 个 token。**关键不在量化本身，而在“分部位”这个决定：**因为各部位是独立 token，就能在 token 层面对动作做替换、扰动、重排（比如把左臂的 token 单独抽掉、或换成另一段动作的左臂 token），这正是第二阶段“自动造数据”能成立的前提。

然后把**运动码本和语言码本合并成一张共享词表**，动作 token 和时长 token 都成了“语言 token”，可以和文字混在一起喂给同一个大语言模型（底座用 Llama3-70B）。一条训练样本长这样：「规划一段动作，

在 <Time> 秒内、以 <State> 这个状态结束。」模型要做的就是自回归地预测下一个动作 token，和语言模型预测下一个词完全一样，目标是最大化对数似然：

$$\mathcal{L}_{\text{LLM}} = - \sum_i \log p(x_o^i \mid x_o^{<i>, x_d)$$

预对齐本身还分两步走：先用**大规模低质量**的视频运动数据（靠算法从视频恢复人体姿态，难免不准）建立初步的语言-运动对齐，再用**小规模高质量**的动捕数据微调、把动作校准到符合真实人体运动学。这个“先广后精”的次序，是后面数据增强消融能成立的关键。

第二阶段，自监督数据增强，这是全篇的灵魂。网上人体视频海量，可**没有文字标注**，没法直接做语言-运动对齐。常见的两条路都不好走：人工标注太贵、规模上不去；用视频大模型自动标注便宜但噪声大，而且它抓不住细粒度的动作细节，描述复杂动作时经常含糊、遗漏甚至出错。

Humanoid-VLA 的答案很反直觉：**不去给动作配文字描述，而是从运动数据自己身上设计自监督任务，自动造出“指令-动作”配对**。既然第一阶段已经把动作拆成可任意操作的部位 token，就能在 token 层面做文章。四类任务：

增强类型	怎么造任务	自动生成的指令（示例）
<Track>	抽出某个关节（如根关节）的运动轨迹	「请让重心沿着 <Track> 这条轨迹移动」
<Time>	取动作时长	「在 <Time> 秒内完成动作」
<Occlusion>	遮挡某个身体部位的 token	「缺失左臂 <Occlusion> 的运动数据，请补全」
<State>	取某一时刻的姿态状态	「规划一段以 <State> 结束的动作」

四类的答案都是完整动作序列。每一类都是“从一段已知动作里挖掉或抽出某种信息，让模型把它补回来”，于是**指令和答案全都来自动作数据本身，完全不需要外部标注**。三个好处：可组合可扩展（<Track> 能和 <Time> 叠起来拼出更复杂的任务，同一条指令还能用 GPT-4 改写成不同说法丰富语言多样性）；吃透了动作的时空结构（这些任务逼模型去学动作内部的时间、空间关系）；输入输出里运动和文字**交错**出现，增强跨模态对齐。靠这套增强攒出的运动-语言交错数据集，规模是前作的 **25 倍**。

第三阶段，视觉条件微调。到这一步才用那批稀缺的“动捕加第一人称视觉”成对数据，关键是要**省**：既要注入视觉，又不能毁掉前面辛苦预对齐好的运动-语言能力。做法是参数高效微调：**复制并冻结**预对齐阶段的 transformer 层（预训练好的知识原封不动保住），新增一个视觉编码器把第一人称图像编码成视觉 token，再在每一层加一个 cross-attention 做融合，其中**语言 token 作 query、视觉 token 作 key / value**。

第四阶段，全身控制器把动作落到机器人。到这里模型产出的还是“人体运动序列”、不是机器人能执行的关节指令。最后由一个**目标条件的强化学习策略**（PPO 训练）把人体运动映射到人形机器人的 24 个关节、输出底层 PD 控制器的目标位置。这一块不是本篇创新，但这个分工很值得记：上层（大语言模型加视觉）负责“该做什么动作”，下层（PPO 全身控制器）负责“怎么在真实机器人上稳稳地做

出来”。

效果如何？最终数据集合并 7 个来源，共 **0.929M 段动作片段、557.3M 帧、7790 小时**。这组构成本身就是论文的论点：

来源	有文字	有动作	片段	帧 / 时长
动捕	有	有	29K	0.3M 帧 / 4.1 h
在线视频	无	有	0.8M	541M 帧 / 7515 h
合成数据	有	有	100K	16M 帧 / 227.7 h

真正的动捕只有 4.1 小时，而无标注在线视频有 7515 小时。人形 VLA 的数据规模，可以靠廉价无标注视频撑起来。

运动生成质量上，标准文本到动作任务里 HumanML3D 的 FID 做到 **0.467**，比扩散式的 MDM 提升 47.5%、比 T2M-GPT 提升 12%；在更难的 Humanoid-S 上多样性也超过 MDM 6%。把动作当 token 让大语言模型自回归生成，质量是过硬的。

数据增强的消融最能说明核心论点：只用高质量动捕、不加视频增强时 FID 是 0.557，把大规模视频运动数据加进来后降到 **0.467**，约 16% 的改善。**海量无标注视频经过自监督增强，真的强化了语言-运动对齐**，而且哪怕拿低质量动捕来微调也能得到可比结果。

物理可行性上，在 IsaacGym 里让控制器跟踪模型生成的轨迹，全局关节位置误差始终低于 40 mm、最优 31.07 mm。真机在 Unitree G1 上覆盖上身、下身、全身、物体交互四类，每任务测 10 次：转向物体、挥手、出拳满分 10/10，踢球、跳过障碍、避障、拿取物体 9/10，与人共舞 8/10。**视觉引导下的接近、定位、踢击、绕障这些“看了才知道怎么动”的任务确实能做出来**，这正是开篇那张对比图追求的从被动复现走向视觉驱动的自主交互。

把身体动作离散成部位 token，和下一节 SONIC 那个“通用 token”是同一个动作 token 化的思路：两边都把连续的全身动作变成一串离散 token，好让大模型像处理语言一样处理它。

3.6 本节小结

大脑这一端四篇工作，回答的是同一个问题的四个层次：语言能不能直驱全身（LangWBC）、驱动信号该做成什么形式（LeVERB 的 latent verb）、除了动手还要会走（WholeBodyVLA 的操作 + 移动双路潜在码）、这些数据从哪来（Humanoid-VLA 的 motion token 预训练）。

把四篇竖着读，会看到一个反复出现的招式：在“想做什么”和“怎么做出来”之间插一个中间表示，让两边各自迭代。差别只在这个表示放在哪、长什么样：

中间表示	它连接什么	直观作用
CVAE 隐变量 (LangWBC)	语言条件与动作风格	给全身动作留出可泛化、可插值的风格空间
latent verb (LeVERB)	视觉语言高层与 50 Hz 控制器	把“场景里该做什么”压成低层能执行的动作意图
操作 / 移动双路潜在动作 (WholeBodyVLA)	无动作视频与 VLA 训练	把两帧画面变化翻译成离散伪动作标签
motion token (Humanoid-VLA)	原始姿态与大模型	把连续身体动作变成语言模型能学习的离散序列

代价也在这里：表示里没有的词，大脑就说不出来。这一句要等下一节把身体那一端讲完才能算完整——因为决定“表示里有哪些词”的，恰恰是底座学过什么。

4 全身底座与基础模型：让身体先会走稳

上一节四篇都把“身体会稳稳动”当成给定条件：LeVERB 的 latent verb 要有个 50 Hz 控制器接住，Humanoid-VLA 的 motion token 要有个执行器把它变成真实关节力矩。这一节就补这一端——先让身体会走稳，再把这个“会走稳”做成通用底座，最后回头和大脑合龙。

4.1 HOVER：立底座

身体这一端最先要解决的，是“控制模式太多、每一种都得单独训一个策略”。HOVER[19] 的洞察是：**全身运动模仿可以当所有控制模式的公共底座**。不管最后要“走快点”还是“把手举到这里”，背后都指向同一件事——让机器人像人一样稳、协调地动。

问题有多碎。人形机器人的卖点是通用形态：既能走路导航、又能搬箱子做 loco-manipulation、又能站在桌前做精细操作。可这些任务对“怎么控制身体”的要求完全不同。导航只关心机器人整体往哪走、走多快，也就是跟踪躯干根部的速度、朝向、高度；桌面操作关心的是上半身关节摆成什么姿态、下半身只要站稳别倒；遥操作或模仿人给的是头、双手这几个关键部位在空间里的三维位置。这三种“你给机器人什么、机器人跟踪什么”的接口就叫**控制模式**。

麻烦在于过去的做法是**每一种模式各训一个专用策略**：ExBody 训一个跟踪上半身关节角的，H2O / OmniH2O 训一个跟踪关键点位置的，HumanPlus 训一个跟踪全身关节的，接口不同、奖励函数不同、训练流程各搭一套。结果是一个为导航训出来的控制器没法切去做操作，上层换个任务、底层就得重训一个控制器。

统一命令空间怎么设计的。图里那组掩码是关键：身体分上下两个区域、两边可以独立选模式，每个维度只能是三种**原子模式**之一——运动学位置跟踪（跟若干关键刚体如头、双手、脚踝的三维位置，遥操

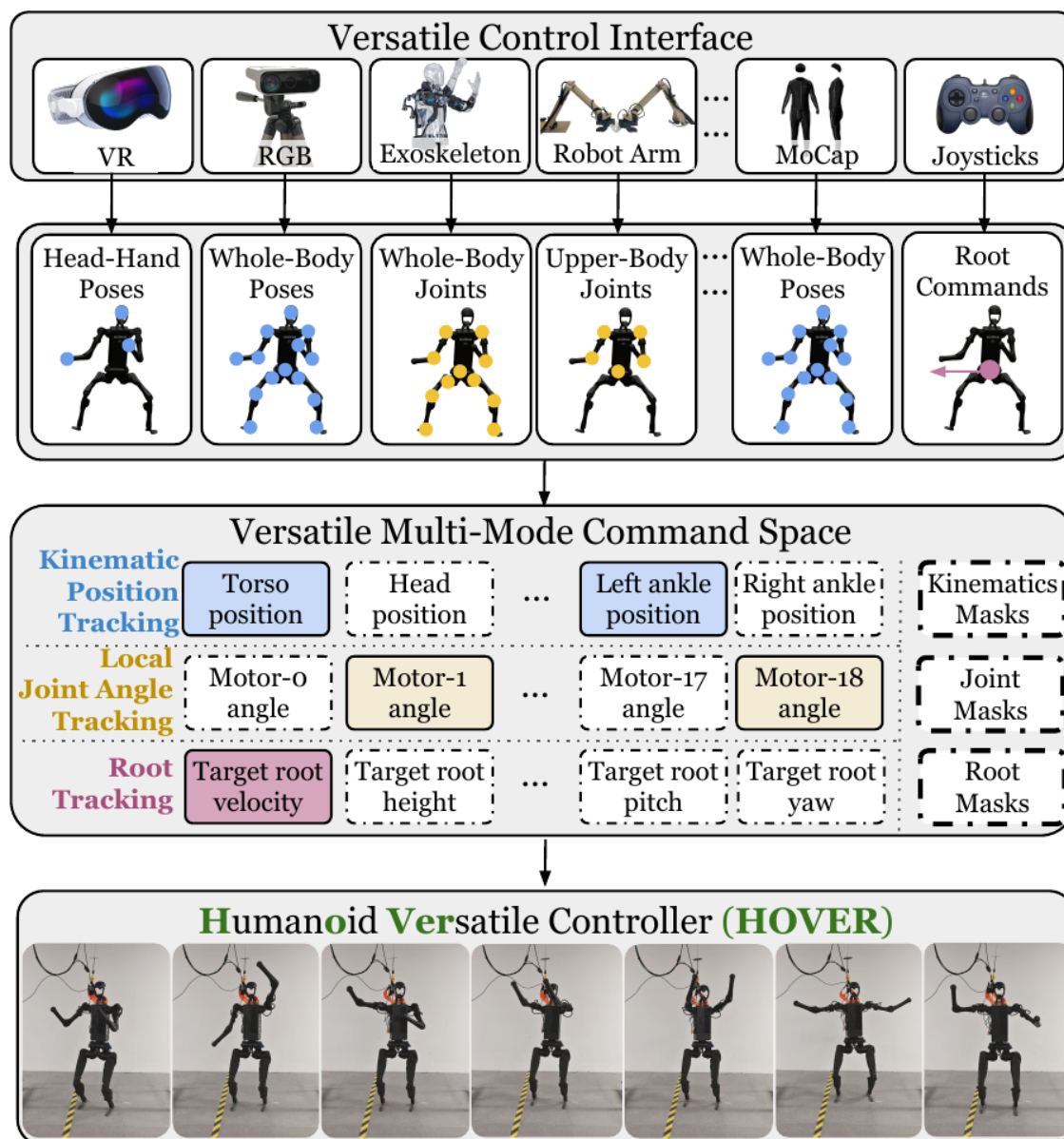


图 13: 自上而下三层: VR / RGB / 外骨骼 / 机械臂 / 动捕 / 手柄各自给出的指令, 先归成“头手位姿”“全身位姿”“上半身关节”这类形式, 再统一表达成一个多模式命令空间——运动学位置跟踪、局部关节角跟踪、根跟踪三种原子模式, 外加一组掩码决定哪些维度这一次要跟 (图片出自参考文献 19)

作最常用)、局部关节角跟踪(直接给定每个关节转到什么角度,要求精确姿态时用)、根跟踪(只管躯干根部的速度、高度、朝向,走路时用)。哪些分量被激活由一个 one-hot 掩码向量决定。

这个空间满足两条准则。**通用性**:能覆盖现有几乎所有配置,还能对接手柄、键盘、动捕、外骨骼、VR 头显。**原子性**:各维度相互独立、可任意组合,完全可以“上半身跟踪双手位置加下半身跟踪根速度”。正因为足够大且原子,过去的工作才能被统一表达:

先前工作	它跟踪的,其实是统一空间的哪个子集
ExBody	上半身关节角 + 根速度
HumanPlus	全身关节 + 根速度
H2O	8 个关键点(双肩、双肘、双手、双踝)的三维位置
OmniH2O	头 + 双手的三维位置

读懂这张表就抓住了 HOVER 的野心:它造的是一个能装下所有模式的容器,而非又一种新模式。

为什么“运动模仿”能当公共底座。要把那么多模式塞进一个网络,直接做几乎不可能,每种模式的输入维度和物理含义都不一样,硬拼会互相打架。HOVER 绕过这个困难靠的是那个洞察:导航也好操作也好,都只是“让身体的某些部位按某种目标运动”的特例;如果一个策略已经学会模仿人类各种全身动作、会平衡会协调四肢,它就掌握了一套通用的运动技能底座。人类动作数据正好提供这种底座,人类的动作本身就是**稳、省力、协调的**,灌进策略,策略就自然获得关于平衡与协调的先验。用的是 AMASS 这个超大规模人类动捕数据集。

第一步是训一个 oracle 老师。它只追求模仿得准、不考虑多模式,并享有一个学生没有的特权:**能看到完整的运动信息**(所以叫 oracle)。网络是个朴素三层 MLP (512 / 256 / 128),用 PPO 训练,输出 19 个关节的目标角度送进 PD 控制器。参考动作要先过一道预处理:把 AMASS 拟合 SMPL 人体模型、对齐人体与 H1 的对应关键点,用梯度下降把关键点匹配到 H1 得到机器人版动作,再在仿真里**筛掉机器人物理上做不出的动作**,得到一份“H1 真能做出来”的可行动作集。

第二步是两层掩码蒸馏,这是真正的创新所在。学生的本体感知和老师不同,它只用真机上便宜可得的量:关节位置与速度、躯干角速度、重力方向,再加动作历史,在过去 25 帧上堆叠给它一个短时记忆。老师能看到“各刚体的全局位置”这类需要外部动捕才知道的量,学生不能。这种**“老师特权、学生平民”**的设定,正是蒸馏(而非直接 RL)能起作用的前提。

学生的指令输入经过两层掩码打残:

$$s_t^{\text{g-student}} = M_{\text{sparcity}} \odot [M_{\text{mode}} \odot s_t^{\text{g-upper}}, M_{\text{mode}} \odot s_t^{\text{g-lower}}]$$

第一层 M_{mode} 决定大方向(比如“上半身跟踪关键点位置、下半身跟踪关节角加根”),上下半身分开选、组合就翻倍。第二层 M_{sparcity} 在大方向里继续抠细节(同样是上半身跟踪位置,这一局只跟双手、不跟头)。每一位掩码都从 Bernoulli(0.5) 独立采样,每个 episode 开始时随机一次、episode 内不变。在 H1 上这两层能组合出约 **1474 万种**模式。

HOVER 高明的地方就在这儿：它不为每种模式单独设计训练，而是用随机遮挡逼一个网络在训练中见识海量模式。学生为了在任何残缺指令下都能模仿老师，只能学会一种“不管你给我什么子集，我都能调动底层运动技能补全成一个稳的全身动作”的通用能力。

蒸馏用 DAgger：让学生在仿真里 rollout 走出自己的轨迹，在每一帧取出对应的老师状态去查询老师、得到它会做的动作，再用监督回归对齐。关键在第一步，rollout 由学生驱动，所以老师纠正的是“学生自己实际会走到的状态”、而不是老师理想轨迹上的状态，这避免了学生犯错后进入老师没教过的状态而失控。最后靠域随机化（随机化质量、摩擦、延迟）迁到真机 H1。

效果如何？四条结论。

在每个专家自己的主场，**HOVER 也不落下风、甚至更强。**把它放进 ExBody / H2O / OmniH2O / HumanPlus 各自的模式下和对应专用策略正面比，同一个 HOVER 在每种模式的 12 项跟踪指标里至少有 7 项胜过专家。这个结论反直觉但重要：“**从一个模仿全身的老师蒸馏出来**”，居然比“**直接为单一模式专训**”还强，原因是 oracle 提供的全身运动先验让学生在任何子模式下都有更扎实的平衡与协调底子。

最能说明问题的是那个对照基线：同样用两层掩码、但不蒸馏、直接用 RL 从头训。结果 HOVER 在 8 种模式乘 4 项指标共 **32/32** 全部跟踪误差更低。这说明多模式统一的功劳**不在掩码本身，而在“从一个已经吃透全身运动学的 oracle 蒸馏”这条路径。**光有掩码、没有好老师，学不出来。

真机上，20 个站立动作的 12 项指标里 11 项胜过专家；更有看点的是**在线突然切换模式**：机器人在行走、转身、后退途中可以从 ExBody 模式直接切到 H2O、从 HumanPlus 切到 OmniH2O 而不摔，还能用 Vision Pro 遥操作、随机遮挡头或手的位置指令仍稳定跟踪。

它没解决的那一半。有一项 HOVER 不一定赢专家：纯**根速度跟踪**。原因是 oracle 聚焦于“未来的运动学位姿”这类二阶信息，会引入固有的跟踪延迟，在“只看速度”这种一阶任务上反而吃亏。这条局限本质上是**老师的能力边界**决定的。而它的规模、泛化、动作多样性也都还有上限：它在一个 19 自由度的 H1 上、靠 AMASS 这一份数据把多模式统一进了一张网，可“动作追踪”这件事能不能**规模化**、从一个能切模式的控制器做成真正通用的**行为基础模型**？下一节的 SONIC 就沿这条路往上走。HOVER 把底座立了起来，SONIC 要把它做大。

最后值得一提的是，这套“oracle 老师 → 残缺指令学生”的蒸馏，和 3.2 节 LangWBC 的 Teacher → CVAE Student 是同一个套路：都靠蒸馏把一个会全身动的控制器逼出来，区别只在 LangWBC 给学生加了语言条件，HOVER 追求把所有控制模式装进一张网。

4.2 SONIC：做大底座

HOVER 证明了底座可以统一，SONIC[20] 接着把它做大。它诊断出人形控制吃不到“规模红利”的卡点不在算法、而在任务选择，于是把**动作追踪**当成那个可规模化的基础任务（逐帧贴近动捕轨迹，监督稠密、不用设计奖励），沿网络规模、数据量、算力三条轴一起放大（论文自己用的词是 supersize），做成一个全身控制的行为基础模型，并暴露出一个干净的**通用 token 接口**。

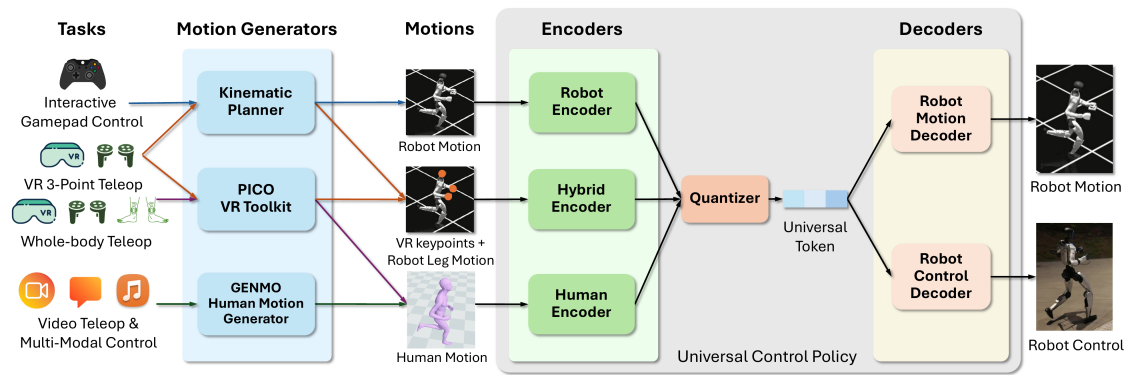


图 14: SONIC 总览：手柄、VR 三点遥操作、全身遥操作、视频与多模态控制四类任务源各自生成动作，经 Robot / Hybrid / Human 三个编码器进入共享 latent，由量化器压成一串通用 token，再分别经动作解码器与控制解码器落成机器人运动和实机控制（图片出自参考文献 20）

为什么要换一种思路。 HOVER 的视角本质上是“把已知的若干模式合并到一起”，模式的清单是人定的，每加一种能力仍然要在指令格式、奖励、训练配置上动手。如果要的是一个真正通用鲁棒、能跑能跳能爬、面对没见过的动作也不摔的全身控制器，而不止于“合并 N 种模式”，逐模式合并这条路就太窄了。

问题出在更底层：**人形控制为什么一直没有享受到“规模红利”**？过去十年大模型靠规模换来了涌现、泛化和鲁棒性，GPT 在两万多张 GPU 上吃下万亿 token，图像与视频生成模型用上千张 GPU 处理数十亿张图。可人形 sim-to-real 控制这边，最强的策略往往还是三层 MLP、几张 GPU、只为单一任务训练。

SONIC 的诊断是卡点不在算法、而在任务选择。locomotion 这类任务靠**逐场景手工设计奖励**，而“向前走得稳”的奖励对“跳一段舞”“从地上爬起来”“精确遥操作”几乎不提供任何有用信号，每加一个能力就要重设一套奖励和目标，这种工作量天然不可规模化。生成式模仿（AMP / ASE / CALM）想用“匹配动作分布”给出统一目标、方向是对的，但它们的判别器式信号在数据集变大变多样时容易 **mode collapse**，数据越多反而越学不动，恰好和“规模换性能”相反。

于是把**动作追踪**当成那个可规模化的基础任务：给定一段人类动捕轨迹让机器人逐帧贴近它，监督信号**逐帧稠密**、不需要人设计奖励；而走、跑、跳舞、与物体交互的动捕数据几十年来已经积累到规模化的体量。**目标统一加数据现成，正是可以做大的那种任务。**

三条轴一起放大。 SONIC 的全名是 Supersizing mOtion tracking for Natural humanoId Control，核心动作就一个“大”字：

轴	从	到
网络规模	1.2M 参数	16M → 42M 参数
数据规模	4M 帧	10M → 22M → 1 亿多帧
算力规模	16 GPU	32 → 128 GPU

推满就是那个最大的模型：128 张 GPU 训约 7 天、21000 GPU · 时、吃下 1 亿帧动作。要注意网络变

大不是把单个 MLP 拉宽，参数主要长在下面那套“多编码器加量化加多解码器”结构上。

数据是一份大规模动捕合集，男女演员均衡，覆盖日常移动、日常活动、手势，以及大量风格各异的格斗动作，源数据约 700 小时；用 GMR 与 PyRoki 重定向到 Unitree G1，再滤掉机器人物理上做不出来的动作（爬楼梯、久坐之类），剩下 **611 小时、1 亿多帧**。

数据的多样性是这套方法的命门，它直接决定泛化能力。训练集覆盖 33 个动作大类、8447 个动作子类：基础与高级移动、舞蹈（嘻哈、拉丁、vogue）、手势、格斗（剑术、武术）、单双手不同高度的物体操作、工具使用（阀门、拉杆、扫帚），还有受伤步态、风格化动作（醉态、僵尸、潜行）、角色扮演。每个动作由多名演员演绎并做镜像，得到成对的左右变体。这份数据相当大的一部分已作为 **BONES-SEED** 公开发布：142220 条标注动作序列、288 小时、522 名演员，附自然语言描述与时间分段标签。

为了能严格评估泛化，测试集被切成三份，这个划分很讲究：**test-content** 包含 182 个**训练时完全没出现过的**动作子类，考的是对全新动作内容的泛化（真正的分布外）；**test-repetition** 是已知动作类型的新表演、新次数，考同类动作不同演绎的鲁棒性；**PHUMA** 是一个公开数据集，用的是不同的重定向流程、来自视频姿态估计，考跨数据集泛化。

通用 token 是最关键的结构创新。图里最值得看的是中间那个瓶颈：手柄、VR 三点遥操作、全身遥操作、视频与多模态控制四类来源，各走各的编码器，量化成同一串**通用 token**，再交给下游解码器落成机器人运动或实机控制。三个编码器分工是：**机器人运动编码器**编码未来若干帧的机器人关节位置与速度（手柄、运动规划器走这条）；**人体运动编码器**编码未来若干帧的 3D 人体 SMPL 关节位置（VR 全身遥操作、视频或文本或音乐生成的人体动作走这条）；**混合编码器**编码当前帧稀疏的上身关键点（头、双手）加未来若干帧的下身机器人运动（VR 三点遥操作走这条，上身实时贴合、下身由规划器补）。多帧输入让策略能预判。

量化用的是 **FSQ** 而不是更常见的 VQ-VAE，理由很实在：

量化方式	问题 / 优势
VQ-VAE	容易 codebook collapse（大批码字闲置）；要额外的 commitment loss 与 codebook EMA 更新；和 PPO 联合优化时麻烦
FSQ	不会 codebook collapse；不需要 commitment loss 与 EMA；提供干净的直通梯度估计， 天然兼容 PPO 联合优化

直通梯度这一点很重要：它让 PPO 的梯度能穿过量化层、反过来塑造编码器的表示。SONIC 在各个模型规模上都没观察到“量化加 RL”带来的训练不稳定。

token 经两个解码器分两路用：**控制解码器**吃 token 加本体感知输出电机命令，仿真训练和真机部署用**完全相同**的输入表示；**运动解码器**只吃 token、重建出机器人运动指令，它是辅助监督、用来打磨 latent 空间。整套损失是 PPO 损失加重建损失加 token 对齐损失加 cycle 一致性损失，端到端联合优化。其中 **token 对齐损失**要求同一段动作无论从机器人、人体 SMPL 还是混合指令进来、产生的 token 都尽量一致，这是三条来源能共享同一个 token 空间的关键约束。

训练用**非对称 actor-critic**：critic 在训练时能看到特权仿真状态（基座线速度、全身连杆位姿、无噪

声观测)，actor 只用部署时拿得到的带噪本体感知和指令。这样 critic 学得准、actor 又能直接迁到真机。再配一整套系统化的域随机化（摩擦、恢复系数、初始关节角、质心位置、随机外力推搡、指令扰动）。

免运行时重定向，这是它工程上最有价值的一点。传统做法里人体动作（来自 VR、视频估计、或 VLA）要驱动机器人，中间必须有一道**重定向**：把人体骨架的动作在线换算成机器人能执行的关节角。这道工序在运行时既费算力又是误差和延迟的来源。

SONIC 把重定向“吸收”进训练：人体编码器把估计出的人体全身动作**直接映射进同一个通用 token 空间**，再由控制解码器驱动机器人。重定向不再是运行时的独立工序，而是早已固化在编码器加量化解码器这套权重里。两个损失项负责这件事：重建损失里有一项专门让“人体 token 解出来的机器人运动”去贴近真实机器人运动，这就是隐式的重定向监督；**cycle 一致性损失**把人体 token 解成机器人运动、再重新编码回 token，要求它和原始机器人 token 一致，保证“人 → 机器人 → 人”这一来一回不丢失动作的本质特征。

于是运行时**不需要任何显式重定向**，同一套策略就能接住 VR 三点遥操作、VR 全身遥操作、视频遥操作，以及 VLA 的输出。一个 token 空间统一了所有这些来源。

效果如何？最值得记的一条是 scaling 规律的形状：把数据、模型、算力三条轴推上去，test-content（分布外）和 test-repetition 上的成功率都**稳定提升**，而**增益主要落在分布外**。这是“规模换性能”在人形控制上第一次被清楚地展示出来。凭这套追踪能力，SONIC 既能做高精度遥操作，也能自然地跑、跳、爬，动作像人。

至于能不能把 VLA 也接到这个 token 接口上，是个很自然的下一步；4.4 节会看到，工程上真正合龙时走的是另一条路。

4.3 GR00T N1：强大脑，但没有腿

GR00T N1[21] 走的是同一个快 / 慢分层思路。它和 LeVERB 谁也没顺着谁——N1 公开更早（2025-03-18），LeVERB 随后（2025-06-16）才给出 latent verb 这个接口，两者是并列的探索。N1 的贡献在于把这套分层做成一个开放、可规模化的基础模型。它要对付的根本困难是：文字和图像有互联网级数据，人形机器人却没有——每条真机轨迹都要一个人实地遥操作才能产生。N1 的主张是把异构数据按“规模”和“本体专属性”组织成一座**数据金字塔**。

学界曾尝试“跨本体”的思路（如 Open X-Embodiment）把许多不同机器人的数据汇到一起凑大，但不同机器人的本体结构、传感器、自由度、控制方式差异巨大，汇起来更像**一堆互不连通的数据孤岛**，而不是训练通才所需的那种连贯大数据集。GR00T N1 的主张是：不要假装这些异构数据是同质的一锅，而是把它们按“规模”和“本体专属性”组织成一座**数据金字塔**，再用一套统一的 VLA 在整座金字塔上协同训练。

模型本身就是这样一个双系统 VLA：一个作为 System 2 的 VLM 负责“想清楚要干什么”，一个作为 System 1 的扩散 Transformer（Diffusion Transformer, DiT，用 flow matching 训练）负责“把动作流



图 15: GR00T N1 的数据金字塔：底层是 Common Crawl、Wikipedia、Reddit 这类网页数据与 Ego4D、EPIC-KITCHENS 这类人类第一视角视频，中层是仿真与合成数据，顶层才是真机数据——越往上越贵、越专属于某一具体本体（图片出自参考文献 21）

畅地动出来”，两者用交叉注意力相连，和 LeVERB 属于同一类快 / 慢分层设计，只是被做成了开放基础模型。

慢系统由预训练的 Eagle-2 担任（SmolLM2 语言模型加 SigLIP-2 图像编码器，已在互联网规模图文数据上对齐过）。三个工程取舍值得记：图像以 224×224 编码、经 pixel shuffle 压缩，每帧得到 64 个图像 token；**不取语言模型的最后一层特征、而是取中间层**（公开的 2B 模型取第 12 层），这一选择既让推理更快、下游策略成功率反而更高，直觉上中间层保留了更多与感知和空间相关的信息、末层则被过度推向纯语言预测；它跑得相对慢，约 10 Hz（在一块 L40 上），这正符合慢系统的定位。

快系统是用 flow matching 训练的 DiT，输出高频闭环电机动作（论文称约 120 Hz）。结构上由交替的自注意力块和交叉注意力块堆叠而成（类似 Flamingo、VIMA）：自注意力块在带噪动作 token 和机器人本体状态之间运作，交叉注意力块负责把视觉-语言条件喂进来。两个系统都是 Transformer，紧耦合、训练时联合优化。公开的 GR00T N1-2B 总共约 2.2B 参数，其中 1.34B 在 VLM 里。

一套大脑吃多种身体的办法很干净：数据金字塔里混着单臂、双臂、人形灵巧手等各种本体，状态与动作维度都不一样，于是**每个本体配一组各自的 MLP 编解码器**把该本体的状态和动作投影到共享嵌入维度，中间那套庞大的 DiT 主干和 VLM 是**所有本体共享的同一套权重**，只有两头薄薄一层编解码器是本体专属的。换新机器人主要就是接一组新的编解码器再微调。

动作是**成块**生成的，一次预测连续 16 步。flow matching 的直觉是“学一条从纯噪声流向真实动作的路径”：训练时把真实动作块按随机插值系数与高斯噪声线性混合得到带噪动作，模型去预测从带噪状态指向真实动作的那个**方向场**；推理时反过来，从纯噪声出发沿模型预测的方向迭代，实践中**4 步**就够。正因为只要 4 步、一次出 16 个动作，采样才够快（在 L40 上采一块只要约 63.9 ms），撑得起高频闭环。和扩散策略放一起记：都是从噪声去噪出动作，但 flow matching 把去噪路径学成一条更直的流，步数更少、更适合实时控制。

怎么用“没有动作”的视频。数据金字塔的底层和中间层有个共同硬伤：网页数据、人类第一人称视频、

神经生成的视频都没有**动作标签**。GR00T N1 的关键创新是给它们补上伪动作，当成额外的本体一起训，两条途径。

潜在动作。训一个 VQ-VAE，编码器吃当前帧和固定窗口后的未来帧、输出一个潜在动作，解码器再用它和当前帧去重建未来帧。它在学“这两帧之间发生了什么动作”，不依赖任何真实动作标签。训好后把编码器当**反向动力学模型**用：给一对前后帧就抽出一个连续的潜在动作当训练标签，用同样的 flow matching 损失去学，只不过把它当作一个独立的“LAPA 本体”。更妙的是 VQ-VAE 是把所有异构数据放在一起训的，于是不同本体（甚至包括人）**共享同一套潜在动作空间**：“右手向左移”这样一个语义动作，在人类、单臂、双臂机器人身上会被映射到相近的潜在动作，这正是跨本体泛化的桥梁。

神经轨迹。这条直接对症“真机数据太贵”。把图生视频的生成模型（基于 Cosmos、CogVideoX、Wan2.1 微调）在自采的 88 小时真机遥操数据上微调，再给定初始帧加新的语言指令，生成出 **827 小时**视频，把真机数据放大约 10 倍。这些生成视频涵盖大量真实世界里没实际采过的**反事实场景**（换个物体、换个放置位置、甚至往杯子里倒液体这种仿真都很难做的画面）。生成视频同样没有动作标签，于是用潜在动作、或一个在真机数据上训好的反向动力学模型来打伪动作，训练时把带伪动作的神经轨迹和真机轨迹按 1:1 混着采样。

中层还有第三类是**仿真轨迹**：用 DexMimicGen 从少量人类示范出发，在仿真里做示范变换与回放自动放大，11 小时内生成了 78 万条，等效于约 6500 小时（连续九个月）的人类示范量。

效果如何？最亮眼的一条是**数据效率**。真机 GR-1 人形上，全量数据下 GR00T N1 平均成功率 76.8%、Diffusion Policy 基线 46.4%。更关键的是低数据档：N1 只用 **10% 的数据**仍能达到 42.6%，而同样只给 10% 数据的 Diffusion Policy 只有 10.2%，差 32.4 个百分点；而且 **N1 用 10% 数据的成绩，只比 Diffusion Policy 用全量还低 3.8 个百分点**。这说明大规模预训练真的把通用先验灌进去了。

仿真三个基准全面领先（每任务 100 条示范下平均 45.0%，高于 Diffusion Policy 的 33.4% 和 BC-Transformer 的 26.4%）。预训练 checkpoint 本身就有用：仅用预训练权重、未针对性微调，在真 GR-1 上做协调双手交接能到 76.6%、把新物体放进没见过的容器能到 73.3%。神经轨迹协同训练确有正收益（RoboCasa 各数据档 +4.2 / +8.8 / +6.8%，真机 8 任务平均 +5.8%），一个有意思的细节是**低数据档时潜在动作略胜反向动力学模型、高数据档时反过来**，因为数据多到一定程度后反向动力学模型推出的伪动作更贴近真实动作。

图里的两条链要分开看：上半条是图像与语言先转成 token 送进 VLM，下半条是机器人状态（关节位置、关节速度、基座位置、末端位姿）单独编码，两者汇进 Diffusion Transformer，由它对一串动作 token 反复去噪，最后出电机动作。**注意 System 1 / System 2 在这里只是架构分工的标签，图上没有给任何频率**——这套系统实际跑多快，得看论文正文里对具体本体报告的数字。

它没解决的那一半，必须说清楚。N1 第一版（公开的 N1-2B）**聚焦的是短时程桌面操作**：真机实验全部是 GR-1 人形坐或立在桌前用双臂和灵巧手做拿放、双手协调、空间推理。论文的局限性一节写得很直白，**长时程的全身移动操作（边走边操作）是明确列出的未来工作**，需要在人形硬件、模型架构、训练语料上一并推进。换句话说，System 1 输出的是桌面操作的手臂动作，**没有腿、不会走、不做全身协调**。文中偶尔出现的 whole-body 字样，指的也只是在仿真里用 minik 做全身逆运动学来做数据重定向。

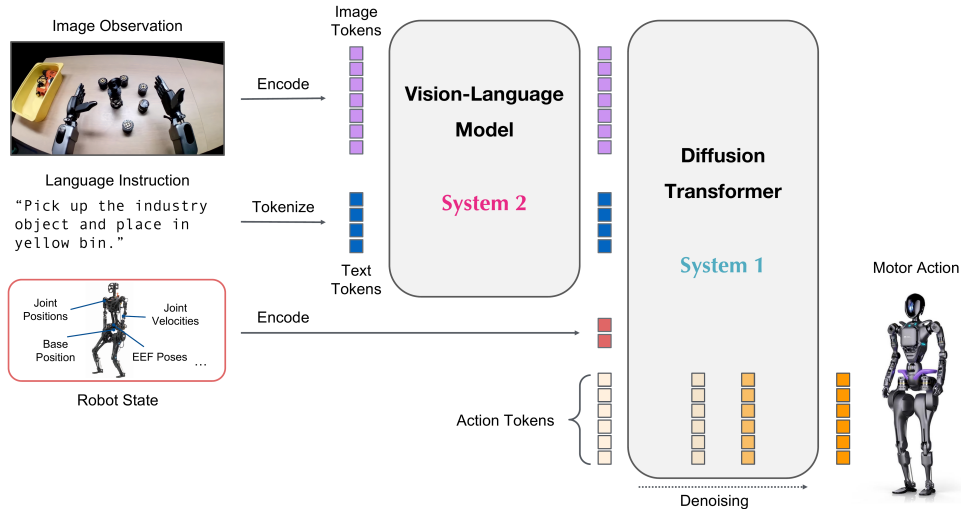


图 16: GR00T N1 推理总览：图像与语言先转 token 交给 VLM，VLM 输出连同机器人状态送进 Diffusion Transformer，迭代去噪生成电机动作（图片出自参考文献 21）

这里要特别警惕一个常见误区：**不要把后续 N1.5 和 SONIC 那些扩展版本的全身能力，安到 N1 第一版头上**。N1 内部确实用了潜在动作，但那是**训练期给无动作视频打伪标注的工具**，不是推理期“高层决策到全身控制器”那种分层接口。N1 v1 就是一个强大的桌面操作基础模型，仅此而已。

正是这个“有强大脑、却没有全身底座”的缺口，定义了下一步要补的东西：一个真正会走、会稳的身体，以及一个把两端接起来的接口。

4.4 GR00T N1.5 / N1.6：用解耦全身控制把两条线合龙

大脑这端（语言、视觉、移动、数据、动作 token）和身体那端（HOVER 的统一命令空间、SONIC 的规模化底座）都齐了，最后一步是把两端拼成一个端到端系统。GR00T N1.5 / N1.6[22] 就是合龙的地方：它是把前面几块积木拼起来的工程节点，不是一篇新论文。合龙前大脑先升级：N1.5 换了更强的 VLM，并加入 FLARE（未来潜在表征对齐）——只让模型对齐未来帧的潜在表征，不生成未来画面，从而能从人类第一视角视频里学到东西 [23]；N1.6 进一步换成 Cosmos-2B 的内部变体，把 DiT 从 16 层扩到 32 层，改成预测相对于当前状态的动作块而不是绝对关节角或末端位姿，并加入更多跨本体数据。

真正的拼法是 **decoupled WBC**——解耦的全身控制（WBC 即 whole-body control，全身控制）。最朴素的想法是让 VLA 直接吐每个关节的角度，但腿的动态平衡是一套高频、对扰动极敏感的控制问题，让擅长“理解任务”的大脑同时学走路既低效又脆弱。GR00T 的选择是把身体沿上下拆开：

下半身交给一个强化学习出来的策略负责走、转、蹲、稳；**上半身**用逆运动学（inverse kinematics, IK）把目标位姿解成关节角 [24]。VLA 大脑（双系统里的 S2，慢、懂任务）只管输出上肢要去哪、身体要往哪走这类高层命令；这套解耦控制器（S1，快、稳、会走路）接过命令，高频执行成电机指令。

这里有一个很容易跳错的地方，得把三个名字的先后与分工捋清。它们是同一条全身控制器线上前后承接的不同阶段，不是三个并列的东西：

- **HOVER** 是这条线的**奠基工作**，第一次证明“多模式统一”这条路走得通；
- **Decoupled WBC** 是这条线**真正被接进 GR00T N1.5 / N1.6** 的那个控制器形态，对应官方 GR00T-WholeBodyControl 仓库里的 Decoupled WBC 条目。也就是说，N1.5 / N1.6 底层用的是它，**不是**直接用 SONIC；
- **SONIC / GEAR-SONIC** 是同一条线**后续更通用的形态**，代表这条接口线继续往通用化走的方向，但它是 Decoupled WBC 之后或并行的演进，**不等于 N1.5 / N1.6 当前实际跑的那一档**。

那个仓库的 README 把 Decoupled WBC 与 GEAR-SONIC Series 列为分开的条目 [24]，这也是最容易读错的一处：**不要把 N1.6 简化成“换成了 GEAR-SONIC”**。更准确的说法是 N1.6 延续 N1.5 的 decoupled WBC 集成方式，同时 VLA 大脑和数据动作空间明显升级。

N1.5 的三处升级值得单独说。第一是 VLM 换成 **Eagle 2.5**，重点是更强的 grounding（把语言里的词对应到画面里的具体物体和位置）和物理解释。差别很直观：一个只会笼统描述画面的 VLM，听到“把右边那个红色的杯子放进抽屉”，它对“右边”“红色”“抽屉”到底指画面里哪一块是糊的；grounding 更强的 VLM 能把每个词精准钉到像素上。最直接的结果是**语言条件控制能力显著增强**，换一句话的指令它真的会换一套行为。

第二是训练加了 **FLARE**（未来潜在表征对齐），这里有个很值得讲清的取舍。一种常见思路是让模型去**预测未来的画面**，以此逼它理解动态。但生成像素是个很重的活，模型大量算力花在画清楚每一根头发丝上，而这些细节对“该怎么动手”毫无帮助。FLARE 换了角度：**不生成未来的画面，只对齐未来的潜在表征**，模型只需让自己内部对“未来会变成什么样”的那个高维向量和真实未来的向量对得上。意图一样（逼模型预判后果），代价小得多。

这个改动带来两个好处。策略性能更好，因为模型被迫学到“我这么动、世界会怎么变”的预判。更关键的是它**解锁了从人类视频学习**：人类操作视频没有机器人动作标签，但它有“未来的画面”；既然 FLARE 只要求对齐未来的潜在表征、不要求动作标签，海量无动作的人类视频就能直接进训练。这正好补上了 4.3 节数据金字塔底层那一块的利用方式。

第三是用 **DreamGen** 造合成数据，在预训练里**新加 12 个动词**。VLA 的能力边界很大程度上是数据边界，训练时没见过“倒”“擦”“按”这类动作，部署时大概率也不会，而真机采集每个新动词都很贵。从数据构成看，DreamGen 不是主菜：真机 GR-1、仿真 GR-1、OpenXE 各约四分之一，DreamGen 与 AgiBot-Beta 各约 9%，**它是用一小块合成数据专门补“真机没采到的新动词”这块短板**。

效果上，在 12 个 DreamGen 任务上 **N1.5 成功率约 38.3%、N1 只有约 13.1%**。这个近三倍的提升说明合成数据加 FLARE 加更强 grounding 这套组合，确实让模型把没真机采过的新动作也学了进去。一句话概括：**N1.5 没改双系统的骨架，但把“看懂场景、听懂指令、会的动作”这三件事整体推高了一档**。

N1.6 改的是大脑而非底座。官方报告页把它定义为 N1.5 的改进版，变化落在 VLA 大脑、动作表示和数据混合上：VLM 换成内部 **Cosmos-2B** 变体（支持更灵活的图像分辨率，并在具身推理任务上训

练); DiT 从 16 层扩到 **32 层**; 移除 N1.5 的 post-VLM 四层 transformer adapter、改为预训练时**解冻 VLM 顶部 4 层**; 多数本体改为预测 **state-relative 动作块**而非绝对关节角或末端位姿; 预训练数据在 N1.5 之外加入更多跨本体数据, 包括 Bimanual YAM、AgiBot Genie-1、Galaxea R1 Pro 仿真, 以及 **Unitree G1 的全身移动操作**。

整条数据流是: 摄像头画面加一句语言指令进来 → 升级后的 VLA 大脑 (S2 侧, N1.5 是 Eagle 2.5 加 FLARE 加 DreamGen, N1.6 进一步换 Cosmos-2B、32 层 DiT、state-relative 动作) → Decoupled WBC 全身控制栈 (S1 侧) 接过高层动作或目标、高频执行成腿加臂加手的电机命令并维持平衡 → 机器人一边走位一边操作。

顺带记一条 4.2 节报过的数字: **SONIC 驱动 VLA 做移动操作, 在若干全身任务上平均约 75% 成功** (苹果入盘、踩踏板开垃圾桶这类要走位加操作配合的任务), 而且让 VLA 预测统一 token 比预测显式人体姿态更平滑、更安全。这说明通用控制器线**能够**服务 VLA 全身任务, 但同样不等于 N1.6 官方系统已经等同于 GEAR-SONIC。

备注: 4.3 节说过 N1 内部也出现过 latent action, 但那是**训练期给无动作视频打伪标签**的工具; SONIC 的通用 token 是**控制器线里用于统一动作追踪接口**的表示。两者用途完全不同, 别混; 也不要把通用 token 直接等同为 N1.6 官方系统的全部接口。

收益是同一个: 两端可以独立迭代、各自规模化。大脑越做越懂任务, 底座越做越稳越通用, 互不拖累。绕了一圈会发现, 这个解耦接口和 3.3 节 LeVERB 的“潜在接口”是同一件事在工程上的落地版本: 从一句话指令到全身电机命令, 中间始终隔着一个干净的中层, 只不过这一层里跑的从一个潜在向量变成了几路可读的分块命令。

4.5 本节小结

身体这一端三篇工作是一路往上垒的: HOVER 把各种控制模式收到同一个全身运动模仿底座上, SONIC 把这个底座做大并给它一个统一的 token 入口, GR00T 一家则从大脑那侧走过来, 到 N1.5/N1.6 才和底座正式接上。

接上的方式, 仍然是上一节那个招式, 只是这次的中间表示由身体这一端定义:

中间表示	它连接什么	直观作用
通用 token (SONIC)	各路指令源与同一个控制解码器	把手柄 / VR / 全身遥操作 / 视频统一成一种内部 token
上下解耦的高层命令 (GR00T N1.5/N1.6 的 decoupled WBC)	VLA 大脑 (S2) 与解耦全身控制器 (S1)	大脑只出“上肢去哪、身体往哪走”, 下肢用 RL、上肢用 IK 各自高频执行

把这一节和上一节的两张表并起来看, 是同一个判断: 让“想做什么”和“怎么稳稳做出来”在一个干净的接口处分工——既把复杂全身动作压进一个可沟通、可泛化的中间空间, 也让大脑与身体能各自独立迭代、各自规模化。

现在可以把上一节那句代价补完了，而且这一笔比前两节的更硬：**接口划在哪，天花板就落在哪**。大脑只能说出接口里有的词，而接口里有哪些词由底座决定——底座学不会的动作，再懂任务的大脑也表达不出来，LeVERB 的潜在 verb 表达不了 WBC 策略没练过的行为，decoupled WBC 的上肢 IK 也做不出需要全身协同发力的动作。同时，一旦接受了这个分层，**端到端联合优化就被放弃了**：两端各自最优不等于整体最优，“走过去姿态会不会妨碍接下来抓取”这类跨层的事，恰恰是 3.4 节那个“面向操作的移动”想补回来的东西。这两节所有工作都在同一条张力上取值——分得越干净，越好规模化，也越难做那些必须跨层才做得成的事。# 5 本讲小结与下一讲

5.1 三条前沿的共同方向

这一讲沿三个方向看了 VLA 的前沿：实时推理（算法 + 系统 + 学习三线合力，让大模型跟上控制频率）、长时记忆（从隐式上下文到显式记忆库，再到记忆 + 想象与长程收口）、人形全身（第 3、4 节的两端——大脑那侧把一句话压成潜在动作意图，身体这侧把会走稳做成通用底座，最后在一个解耦接口处合龙）。

把它们放在一起，会看到一个共同方向：**VLA 正在从“单帧、桌面、单臂”走向“高频、长程、整身”**。快、久、全身，分别在时间的三个尺度上给 VLA 松绑——一次推理的时间、一个任务的时间跨度、一个身体的空间范围。

更值得记住的是这三条线用的是**同一个招式**：把一条原本要一口气走完的链路，在中间切开一刀，让两边按各自的节奏跑。

- 实时那条线切的是**时间**：RTC 把一个动作块切成冻结段 / 软过渡段 / 自由重画段，FASTER 沿时间视界把去噪步数切成“近端一步、远端多步”，V1 干脆把大模型和动作专家切成两条频率不同的并发回路。
- 记忆那条线切的是**信息**：决策依据从“单帧”切成“工作记忆 + 可检索的记忆库”，当下和历史各存各的、各有各的更新节奏。
- 人形那条线切的是**职责**：懂任务的大脑和会走路的身体切开，中间只留一个潜在 verb 或一组分块命令。

这三刀落在链路的三个互不相干的维度上——时间、信息、职责——这就是“同一个招式”这句话的实质。而接缝同时也是账单的所在：切开之后，就再也没有一个模块能看见全局。这是本讲反复出现的那对取舍，也是判断下一篇 VLA 前沿论文的起点。

5.2 读一篇 VLA 前沿论文，先问四句

开篇说这一讲要给你一把尺子。把上面这些教训收成四个问题，遇到新论文按顺序问一遍，多数时候够用了：

1. **它压的是哪一段？**看到“N 倍加速”先找分母。1.4 节那个“10×”量的是去噪采样，把 22.4 ms 的 prefill 算进来就只剩三倍出头。加速比不写清楚起止点，就等于没写。

2. **它加的东西活在训练期还是推理期？**同样叫 latent，GR00T N1 的潜在动作只是训练期给无动作视频造监督的工具，推理时并不存在；LeVERB 的 latent verb 是运行时高层与低层之间真正传输的东西。一个不占推理预算，一个占。
3. **收益是在什么任务上量的？**2.8 节那三个数——一般任务约 9 个百分点、依赖记忆和依赖想象的任务约 26 和 28 个百分点——说明同一个方法换一批任务收益能差三倍。先看它的基准是不是专为此能力造的。
4. **它把哪一层的活推给了谁？**分层方案都会把难题挪个位置而不是消灭它：大脑不学走路，是因为底座学了；VLA 不预测关节角，是因为 IK 和 RL 策略接住了。**问清楚接住的那一方能力边界在哪，就知道整个系统的天花板在哪。**

5.3 下一讲：换范式进入强化学习

到这一讲为止，从第 8 讲到第 13 讲，我们走的都是**模仿学习**这条主线：从示教数据里学一个策略，无论是 ACT、扩散、流匹配，还是各种 VLA，本质都是“模仿专家怎么做”。它的天花板也很清楚：**纯离线的行为克隆一旦走到示教没覆盖的状态，就没有东西告诉它该怎么走回来**。这不是模仿学习的宿命——DAgger、交互式模仿、数据聚合，以及专门补采纠正与恢复数据的做法，都能把“走错了怎么回来”学进策略里，第 16 讲的人在环方法走的就是这一路；但它们都得额外掏出交互机会或额外的数据，纯离线的那一版掏不出来。

下一讲开始换一个范式：**强化学习**让机器人在环境里试错、用奖励塑造行为，去触碰模仿学习够不到的地方。我们会从最朴素的 REINFORCE 出发、一路升级到 PPO，让宇树 G1 先学会走路，最后再把同一套 PPO 原样搬到动作跟随上。

参考文献

著录遵《GB/T 7714—2015》顺序编码制，正文标记 [N] 按首次引用先后编号。圆括号内为 arXiv v1 提交日期（逐条对 arXiv abs 页的 submission history 核过），方括号内为引用日期 2026-06-26。

- [1] BLACK K, GALLIKER MY, LEVINE S. Real-Time Execution of Action Chunking Flow Policies[EB/OL]. (2025-06-09)[2026-06-26]. <https://arxiv.org/abs/2506.07339>.
- [2] LU Y, LIU Z, FAN X, et al. FASTER: Rethinking Real-Time Flow VLAs[EB/OL]. (2026-03-19)[2026-06-26]. <https://arxiv.org/abs/2603.19199>.
- [3] MA Y, ZHOU Y, YANG Y, et al. Running VLAs at Real-time Speed[EB/OL]. (2025-10-30)[2026-06-26]. <https://arxiv.org/abs/2510.26742>.
- [4] YANG C, HU Y, MA Y, et al. Realtime-VLA V2: Learning to Run VLAs Fast, Smooth, and Accurate[EB/OL]. (2026-03-27)[2026-06-26]. <https://arxiv.org/abs/2603.26360>.

- [5] NIU J, GU K, ZHAO Y, et al. Realtime-VLA FLASH: Speculative Inference Framework for Diffusion-based VLAs[EB/OL]. (2026-05-13)[2026-06-26]. <https://arxiv.org/abs/2605.13778>.
- [6] LIU Y, YU H, ZHAO J, et al. Learning Native Continuation for Action Chunking Flow Policies[EB/OL]. (2026-02-13)[2026-06-26]. <https://arxiv.org/abs/2602.12978>.
- [7] TORNE M, TANG A, LIU Y, et al. Learning Long-Context Diffusion Policies via Past-Token Prediction[EB/OL]. (2025-05-14)[2026-06-26]. <https://arxiv.org/abs/2505.09561>.
- [8] KOO M, CHOI D, KIM T, et al. HAMLET: Switch your Vision-Language-Action Model into a History-Aware Policy[EB/OL]. (2025-10-01)[2026-06-26]. <https://arxiv.org/abs/2510.00695>.
- [9] GUO X, JIANG C, KIM HB, et al. Chameleon: Control-Indexed Prospective Memory for Visuomotor Manipulation[EB/OL]. (2026-03-25)[2026-06-26]. <https://arxiv.org/abs/2603.24576>.
- [10] SHI H, XIE B, LIU Y, et al. MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation[EB/OL]. (2025-08-26)[2026-06-26]. <https://arxiv.org/abs/2508.19236>.
- [11] LI Z, HU B, SHAO R, et al. Global Prior Meets Local Consistency: Dual-Memory Augmented Vision-Language-Action Model for Efficient Robotic Manipulation[EB/OL]. (2026-02-22)[2026-06-26]. <https://arxiv.org/abs/2602.20200>.
- [12] SHI H, LI W, XIE B, et al. MemoryVLA++: Temporal Modeling via Memory and Imagination in Vision-Language-Action Models[EB/OL]. (2026-06-08)[2026-06-26]. <https://arxiv.org/abs/2606.09827>.
- [13] HU Y, ZHANG J, LUO Y, et al. BagelVLA: Enhancing Long-Horizon Manipulation via Interleaved Vision-Language-Action Generation[EB/OL]. (2026-02-10)[2026-06-26]. <https://arxiv.org/abs/2602.09849>.
- [14] FAN Y, DING P, BAI S, et al. Long-VLA: Unleashing Long-Horizon Capability of Vision Language Action Model for Robot Manipulation[EB/OL]. (2025-08-27)[2026-06-26]. <https://arxiv.org/abs/2508.19958>.
- [15] SHAO Y, HUANG X, ZHANG B, et al. LangWBC: Language-directed Humanoid Whole-Body Control via End-to-end Learning[EB/OL]. (2025-04-30)[2026-06-26]. <https://arxiv.org/abs/2504.21738>.
- [16] XUE H, HUANG X, NIU D, et al. LeVERB: Humanoid Whole-Body Control with Latent Vision-Language Instruction[EB/OL]. (2025-06-16)[2026-06-26]. <https://arxiv.org/abs/2506.13751>.
- [17] JIANG H, CHEN J, BU Q, et al. WholeBodyVLA: Towards Unified Latent VLA for Whole-Body Loco-Manipulation Control[EB/OL]. (2025-12-11)[2026-06-26]. <https://arxiv.org/abs/2512.11047>.
- [18] DING P, MA J, TONG X, et al. Humanoid-VLA: Towards Universal Humanoid Control with Visual Integration[EB/OL]. (2025-02-20)[2026-06-26]. <https://arxiv.org/abs/2502.14795>.

- [19] HE T, XIAO W, LIN T, et al. HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots[EB/OL]. (2024-10-28)[2026-06-26]. <https://arxiv.org/abs/2410.21229>.
- [20] LUO Z, YUAN Y, WANG T, et al. SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control[EB/OL]. (2025-11-11)[2026-06-26]. <https://arxiv.org/abs/2511.07820>.
- [21] NVIDIA, BJORCK J, CASTANEDA F, et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots[EB/OL]. (2025-03-18)[2026-06-26]. <https://arxiv.org/abs/2503.14734>.
- [22] NVIDIA GEAR Lab. GR00T N1.6: An Improved Open Foundation Model for Generalist Humanoid Robots[EB/OL]. [2026-06-26]. https://research.nvidia.com/labs/gear/gr00t-n1_6/.
- [23] NVIDIA GEAR Lab. GR00T N1.5: An Improved Open Foundation Model for Generalist Humanoid Robots[EB/OL]. [2026-06-26]. https://research.nvidia.com/labs/gear/gr00t-n1_5/.
- [24] NVIDIA GEAR Lab. GR00T Whole-Body Control: decoupled WBC models used in GR00T N1.5 / N1.6 and GEAR-SONIC[CP/ OL]. [2026-06-26]. <https://github.com/NVlabs/GR00T-WholeBodyControl>.